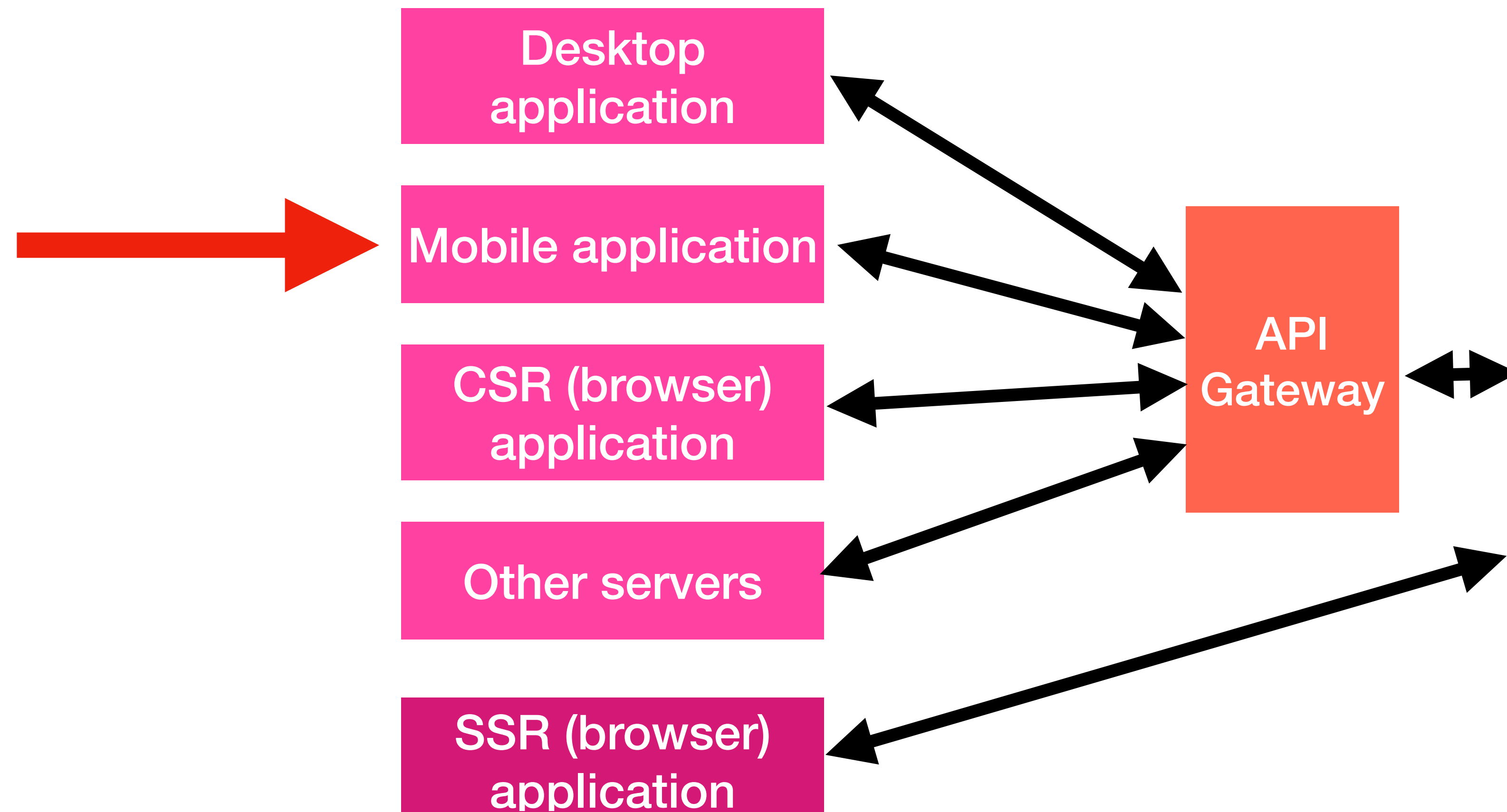


Internet Applications Design and Implementation

Week 10 - Mobile apps.

Client Types



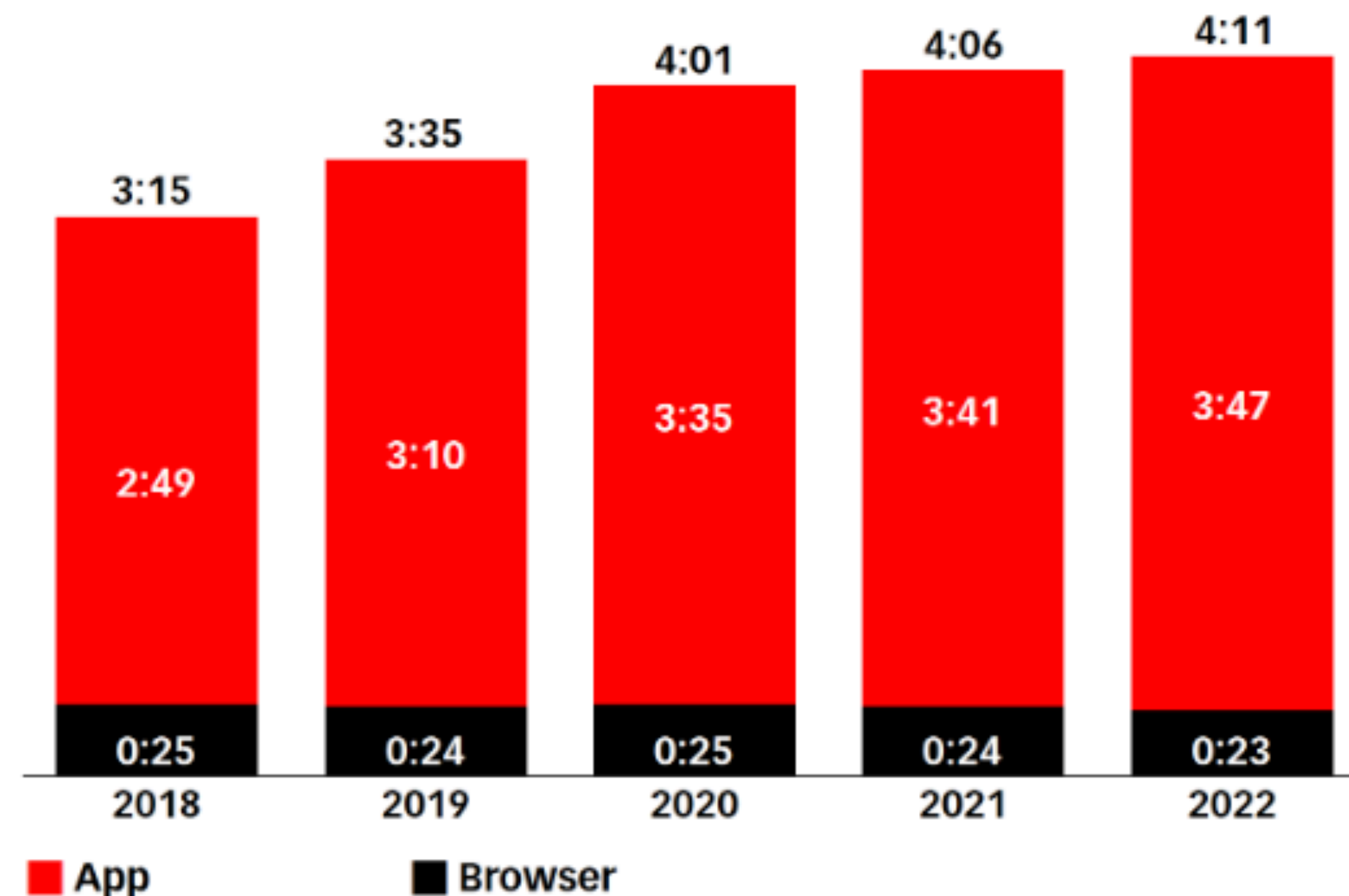
Mobile apps



Mobile apps

Mobile Internet: Average Daily Time Spent in the US, App vs. Browser, 2018-2022

hrs:mins per day among population



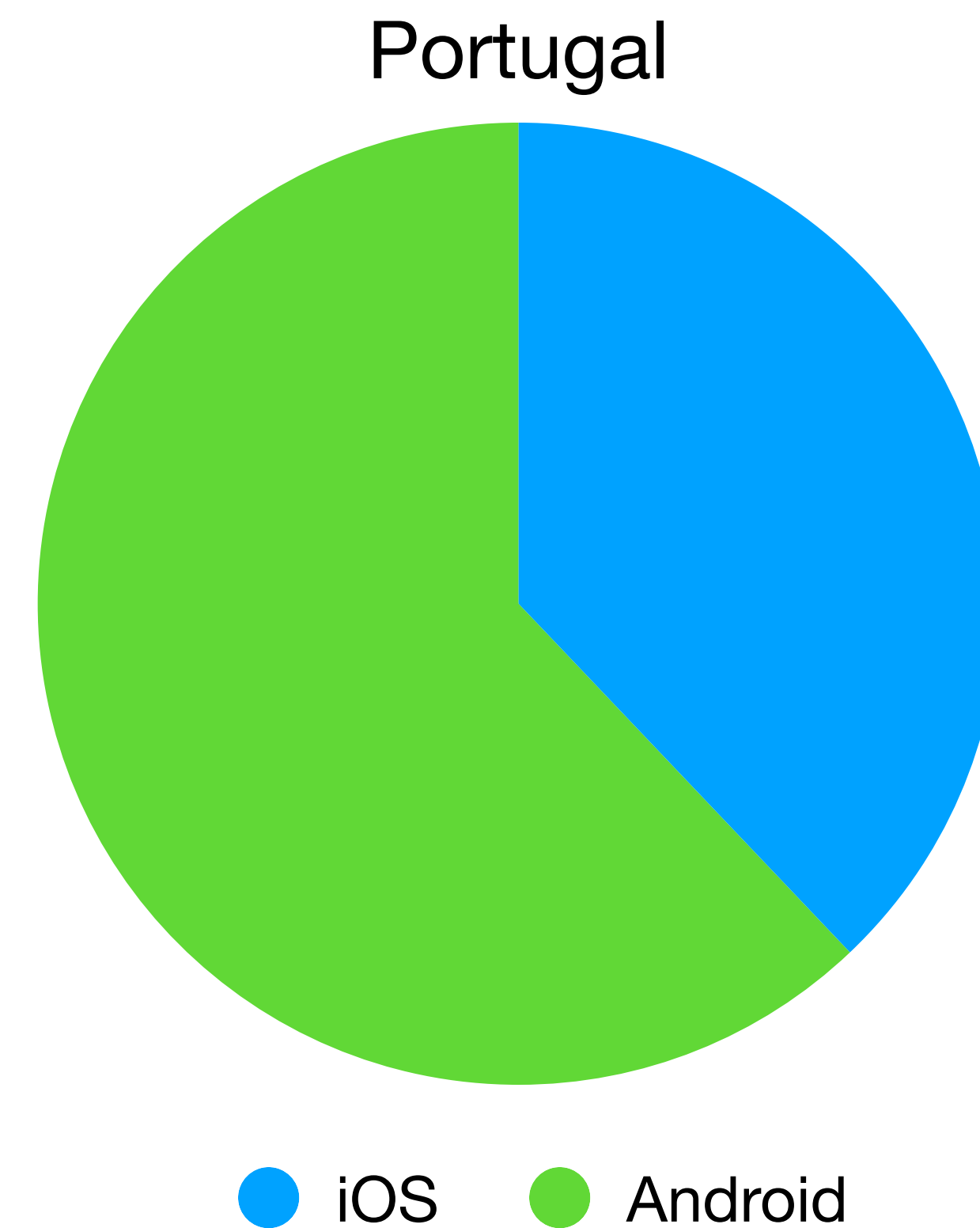
Note: ages 18+; includes all time spent with internet activities on mobile devices, regardless of multitasking; numbers may not add up to total due to rounding

Source: eMarketer, April 2020

On mobile devices, users prefer “apps” than the browser

Mobile apps

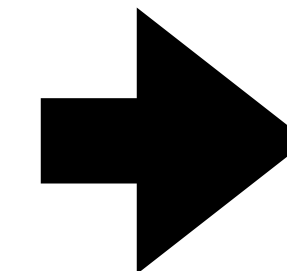
Two operating systems account for 99.9% of the market



Mobile apps

Each OS has its own development stack

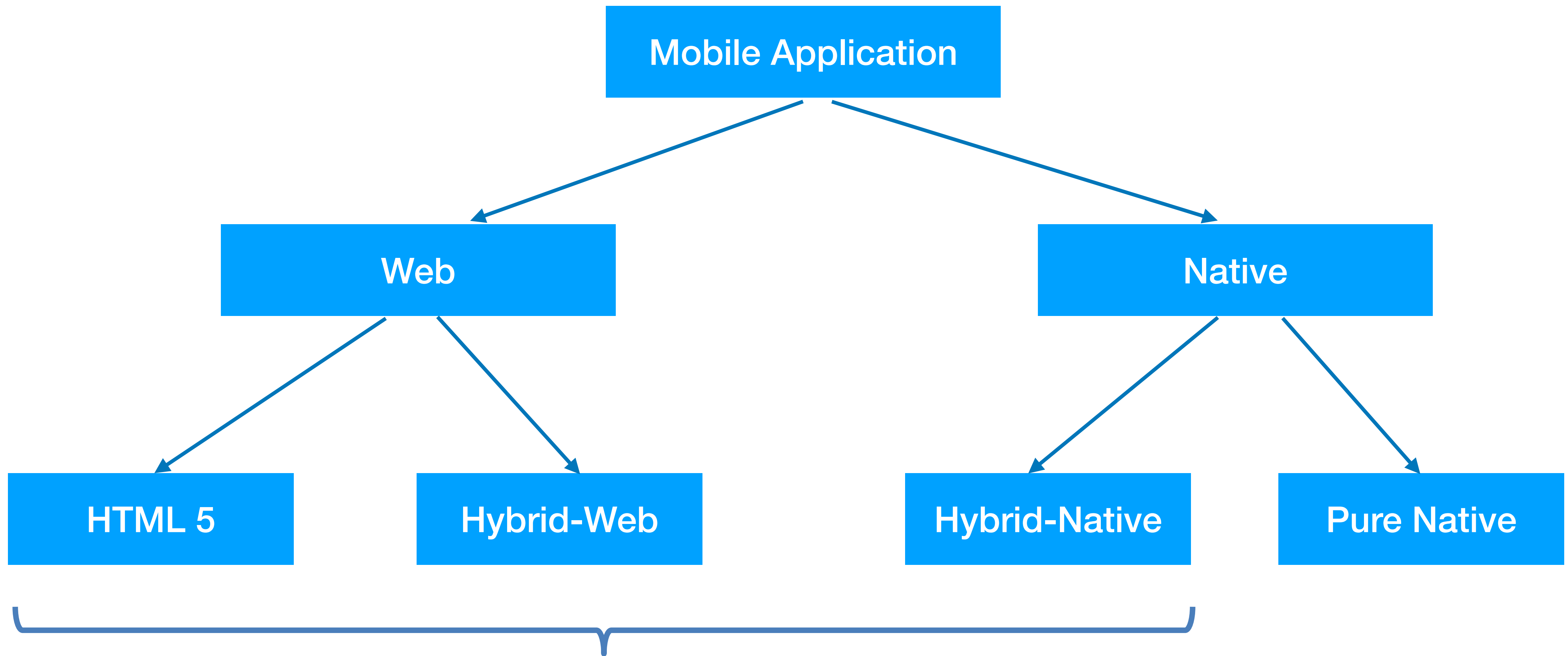
- iOS
 - Objective-C / Swift
 - iOS SDK / Xcode / macOS (*requires a physical Mac machine*)
- Android
 - Java / Kotlin
 - Android SDK / Android Studio / Windows, macOS, Linux



Usually implies dedicated developers/teams for each OS

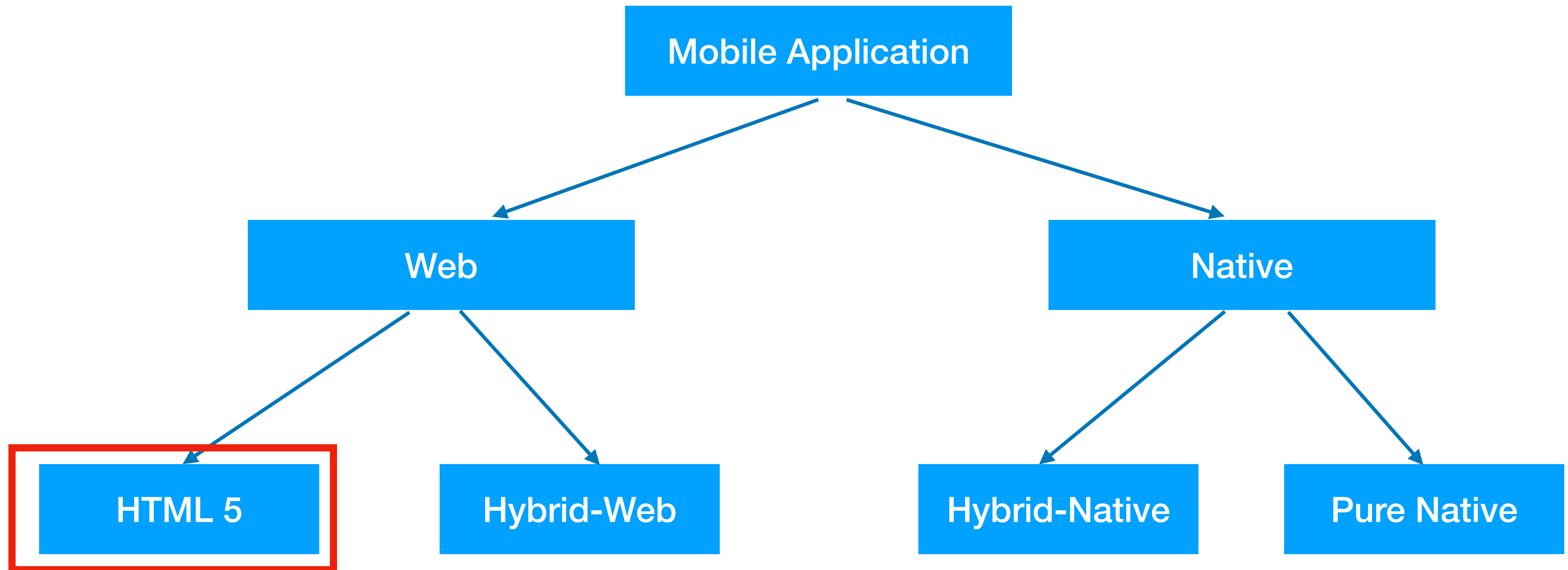
Development costs are duplicated

Mobile apps



Cross-platform, develop just once for both OSes

Mobile apps



Mobile apps

HTML 5

All smartphones have a browser, capable of rendering complex web applications (*)

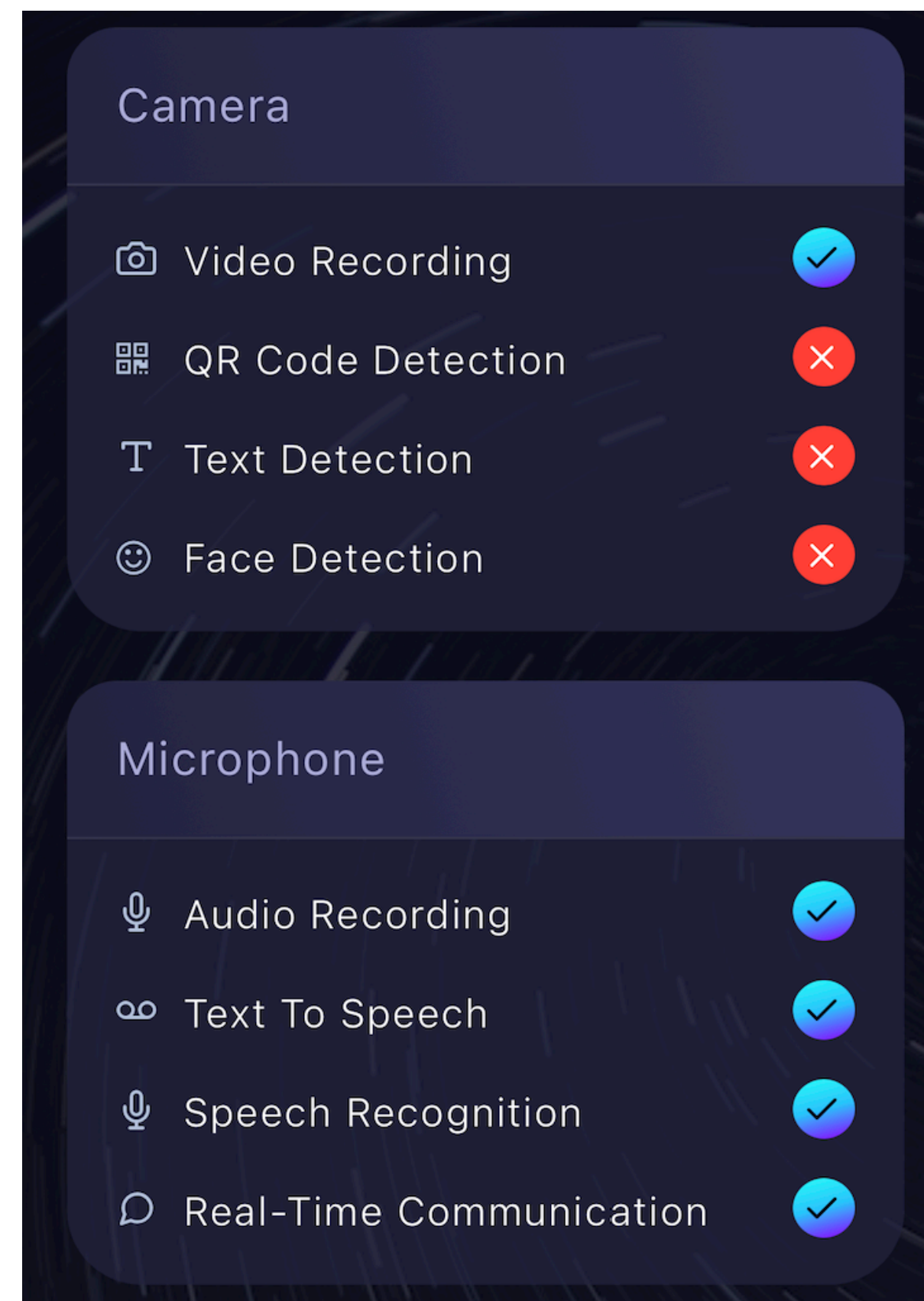
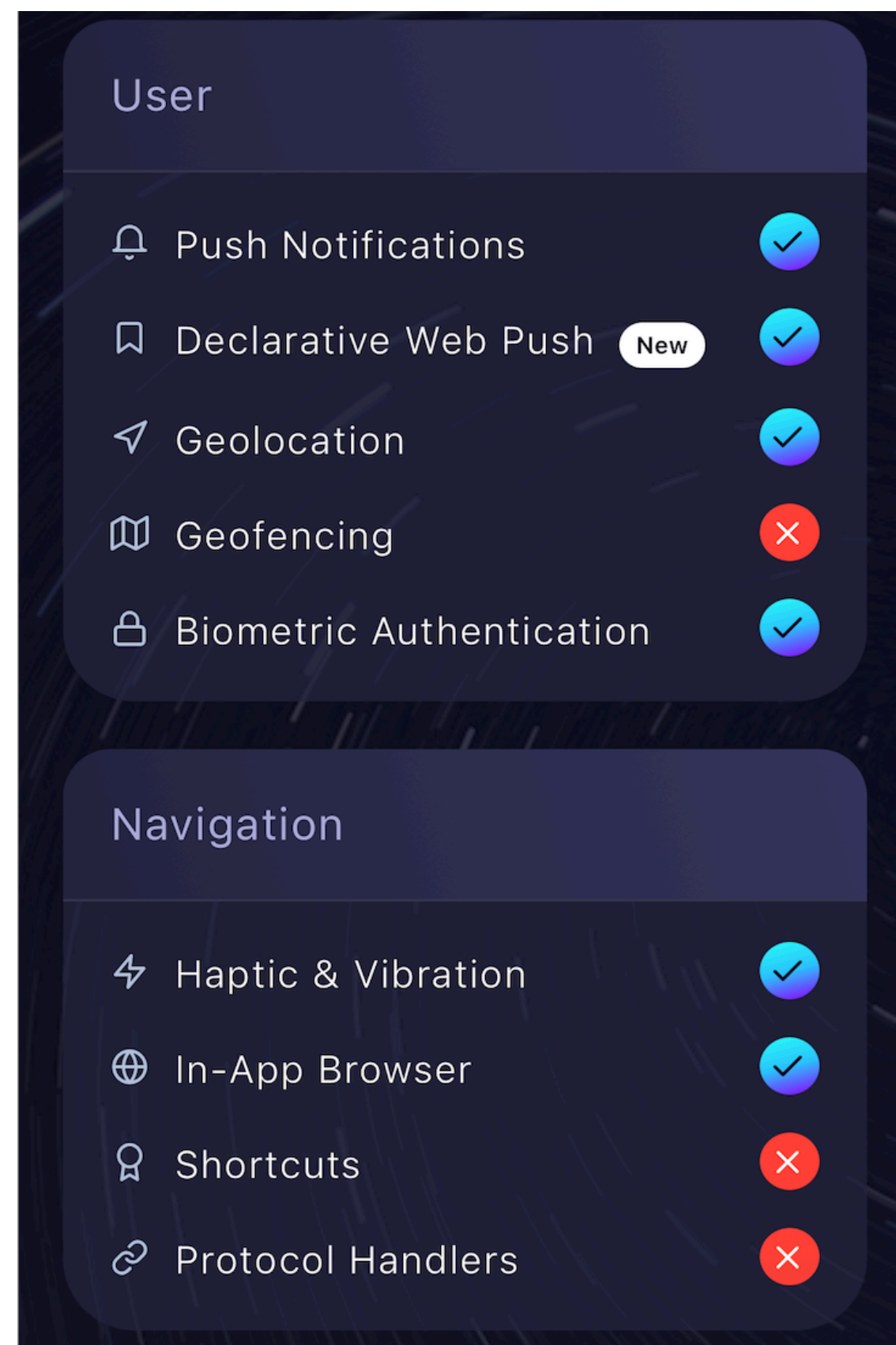
On top of that, they have (limited) access to specific device features:

- camera
- geo-location
- sensors (accelerometer, compass, ...)
- contacts
- ...

(*) Possibly CSR applications

Mobile apps HTML 5

<https://whatwebcando.today/>



Mobile apps

HTML 5

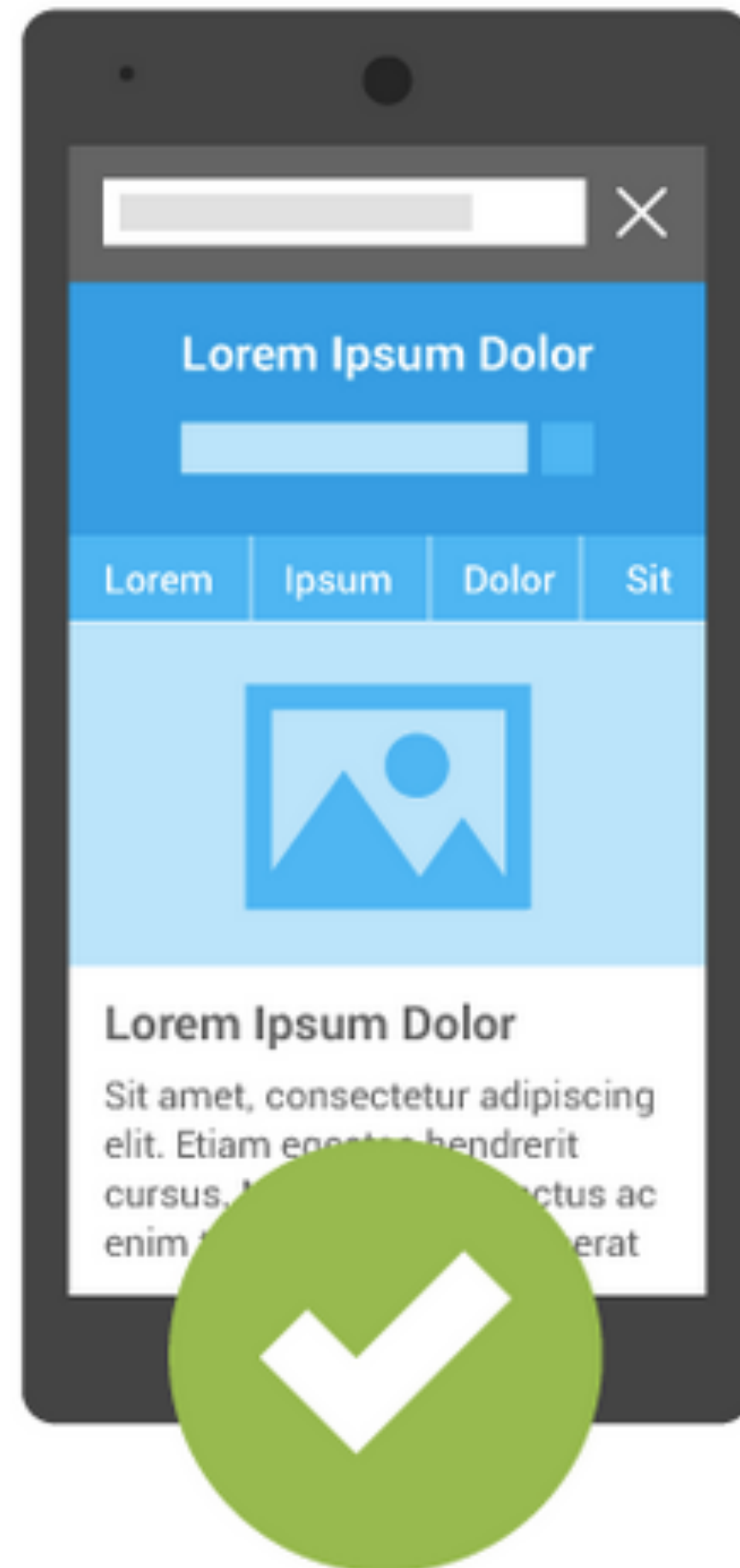
Sites designed for desktop (large screens) have to be adapted for smartphones

Google gives more relevance to **mobile-friendly** pages (specially if you access through a smartphone)



Mobile apps

HTML 5



Mobile-friendly

- Ensure all text is readable at the default zoom level
- Make buttons and interactive elements large enough to tap comfortably and avoid accidental touches (“fat finger” issues)
- Avoid horizontal scrolling; content should fit within the screen width at all times

Mobile apps

HTML 5

3 options for implementing mobile-friendly applications

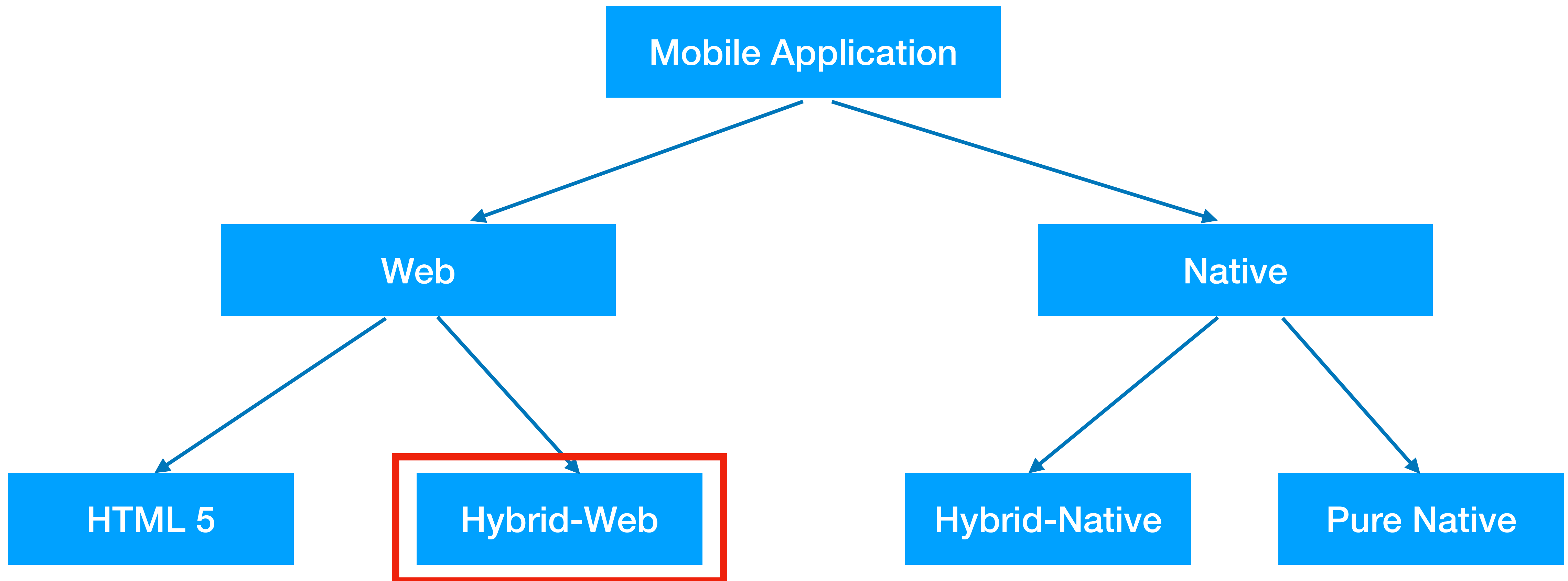
- **Responsive web design** - same page for desktop and mobile - use css to adjust the layout to the screen dimensions
- **Dynamic serving** – same url returns different pages based on the device that is accessing
- **Dedicated mobile URL** – different URL from the base (desktop) url - e.g., www.techmeme.com (desktop) - m.techmeme.com (mobile)

Mobile apps

HTML 5

	Advantages	Disadvantages
Responsive Web Design	A single codebase adapts to different screen sizes	Limited ability to deliver device-specific experiences (e.g., Android vs. iOS)
Dynamic serving	Complete control of the page that is returned to each device	Requires maintaining different pages for each device
Dedicated mobile URL	User can choose how he wants to see the application (e.g., see the mobile version in desktop)	Not a single URL for the application

Mobile apps



Mobile apps

Hybrid-Web

Users tend to prefer *installable* mobile apps over browser-based access, even when the website is easily reachable through links/bookmarks.

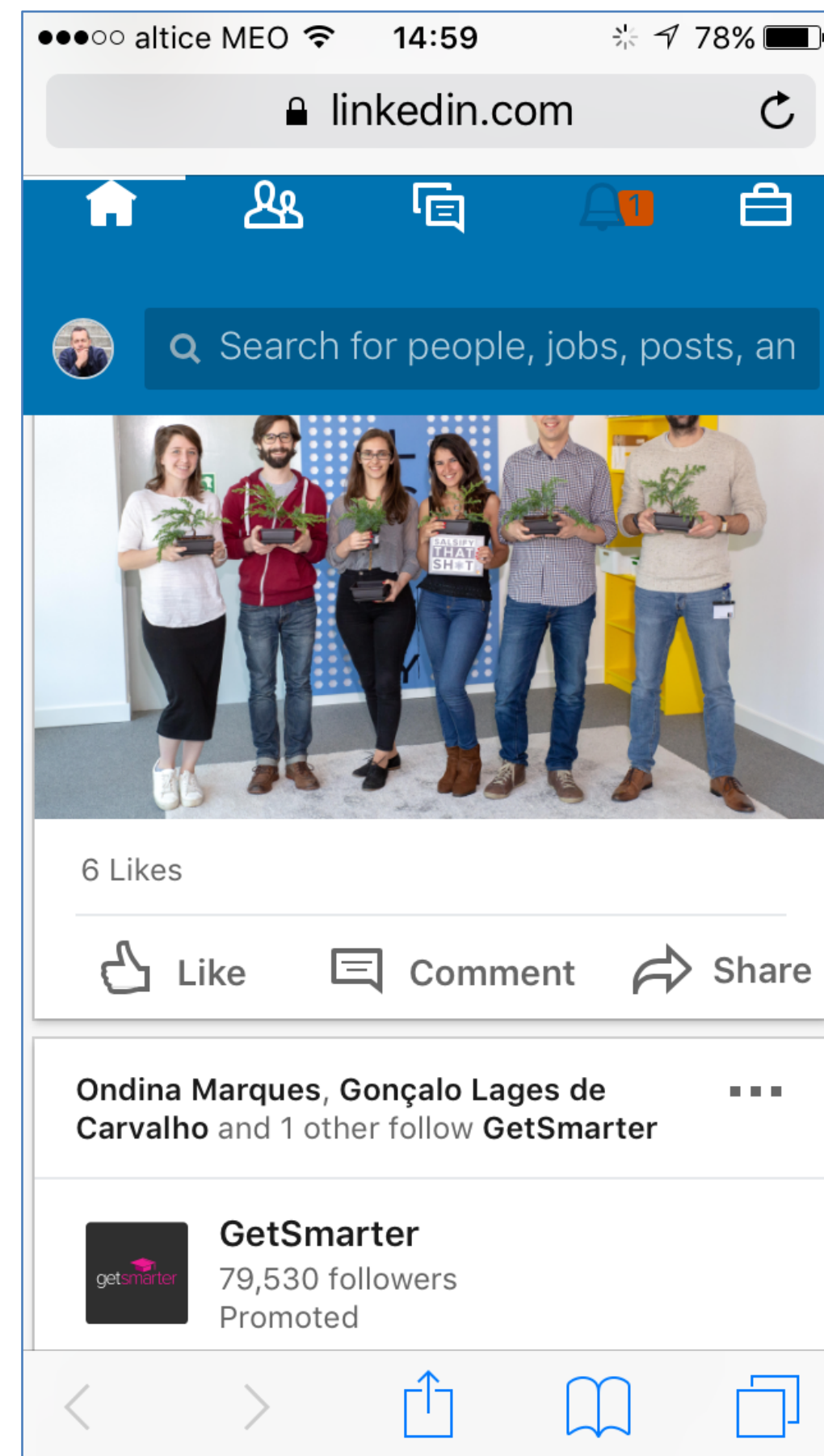
Installable means it is available in the store (app store / google play), not necessarily fully native

What if we wrap the browser into an installable app?

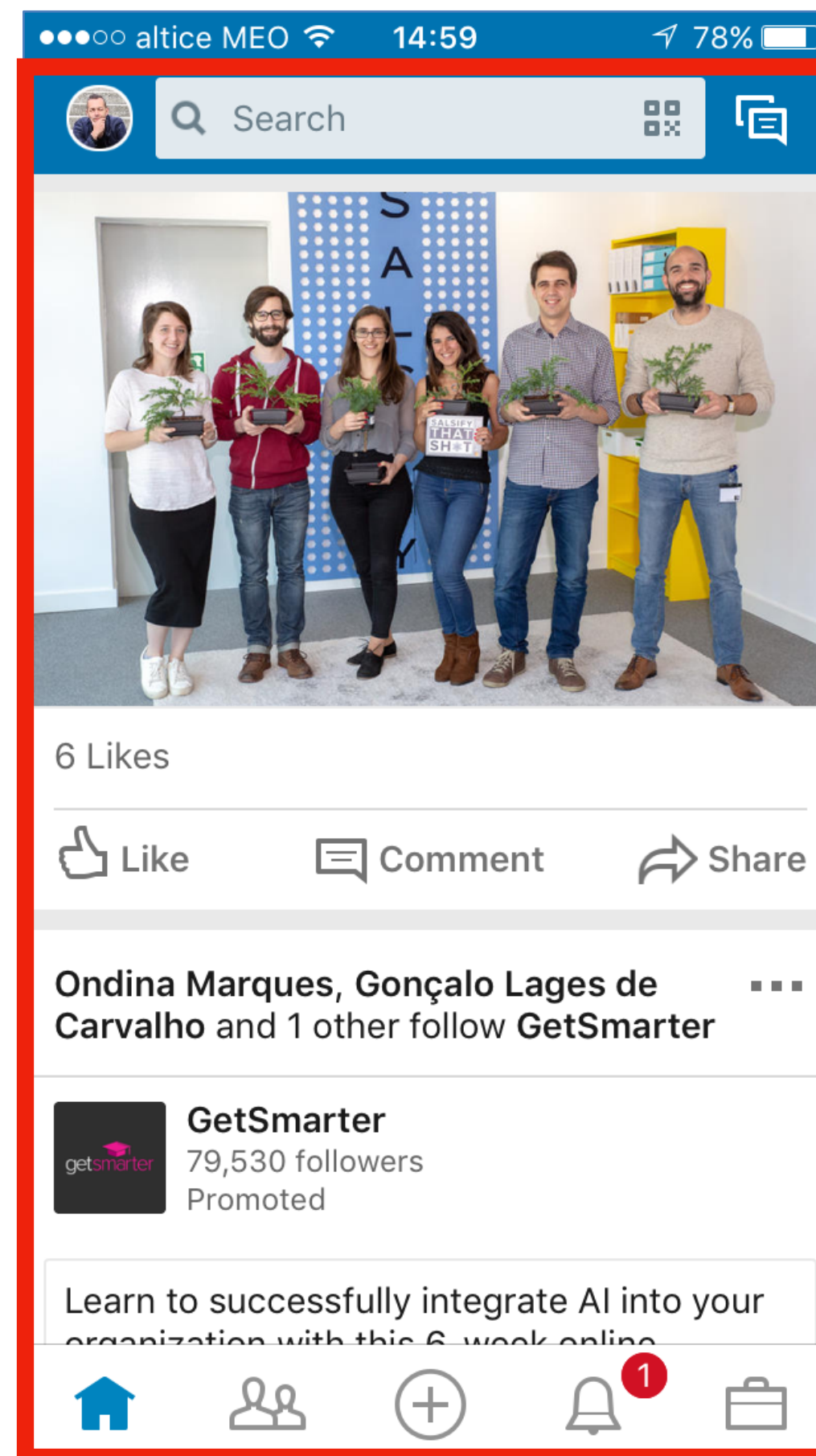
Mobile apps

Hybrid-Web

Browser



Hybrid-Web



A native app with a single visual element:

WebView (fullscreen browser)

Everything is still happening in the browser but the user doesn't notice it.

Mobile apps

Hybrid-Web

Poor man's hybrid-web app in Android

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/container"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    tools:ignore="MergeRootFrame">

    <WebView
        android:id="@+id/activity_main_webview"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

</FrameLayout>
```

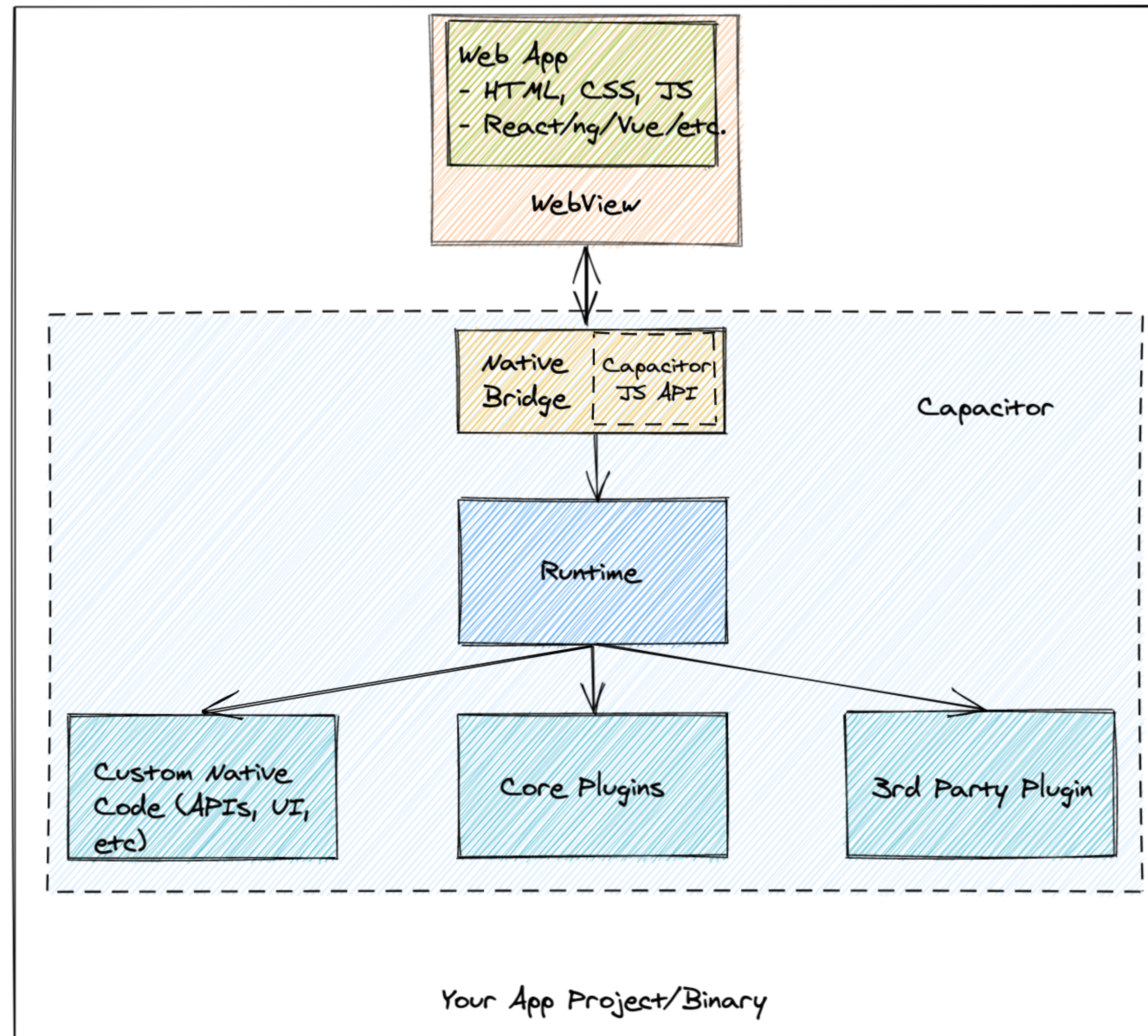
mWebView.loadUrl("file:///android_asset/www/index.html");

Mobile apps

Hybrid-Web

Capacitor

- developed by Ionic
- extends the WebView with native plugins that provide access to advanced device capabilities



Mobile apps

Hybrid-Web

Ionic

- leading hybrid-web framework
- use modern CSR frameworks such as React or Angular
- provides custom CSS to achieve a semi-native look
- integrates with Capacitor



A strong option for teams with existing web frontend expertise

However, it is constrained by the fact that it runs inside a “browser” (not a native look&feel)

Mobile apps

Hybrid-Web

Offline ability

- Users expect installable apps to be able to launch and operate (although limited) in offline



Ionic enables offline mode via two options:

Option 1 - Bundle all the HTML/CSS/JS along with the app

Option 2 - Use Service Workers (*) and caching strategies

(*) Javascript process that runs in the background (even if the browser is closed) and enables offline support, background sync, push notifications, etc...

Mobile apps

Hybrid-Web

Live updates

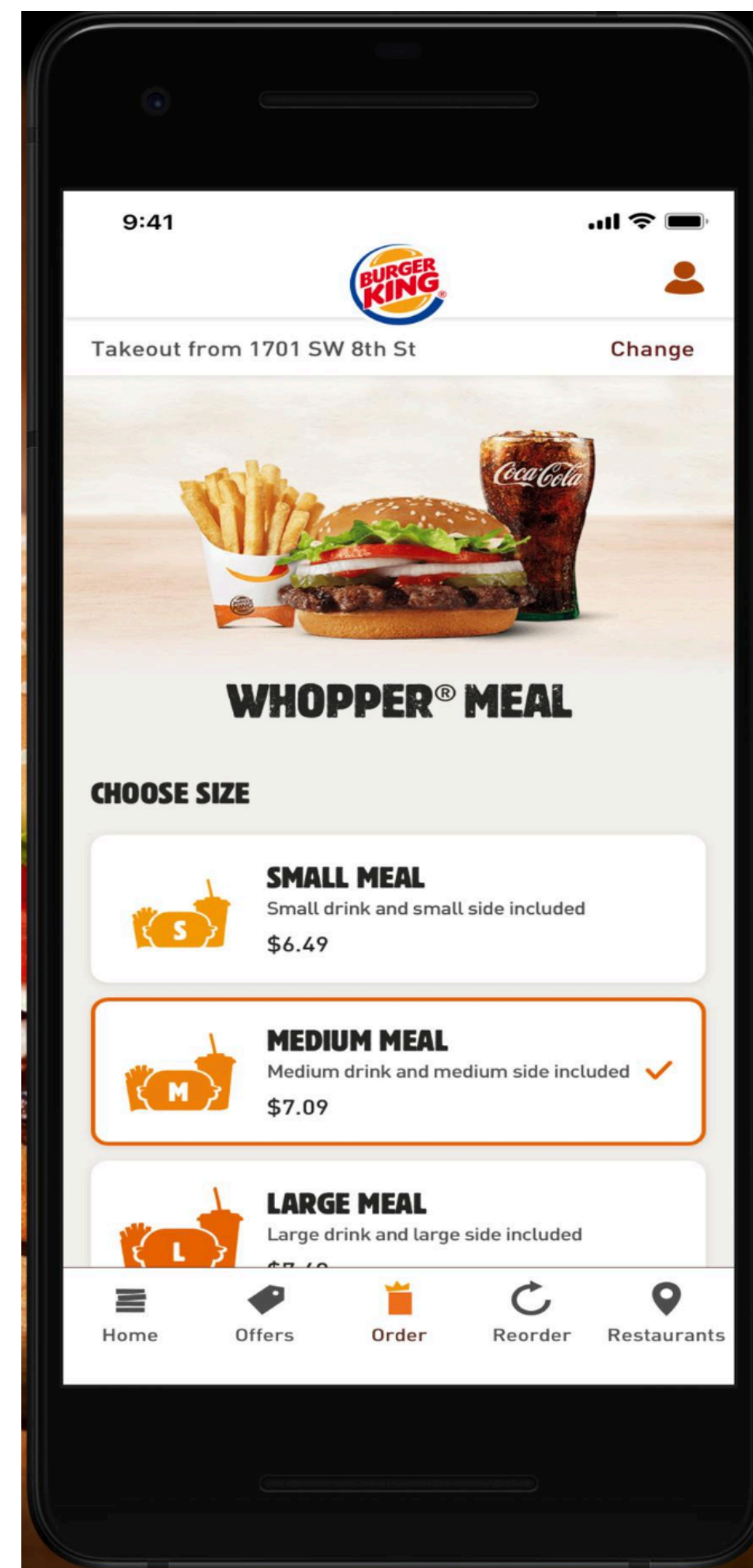
- Ionic Appflow allows updating an application's appearance and behavior without needing to publish a new binary to the app stores. (paid service)

Mobile apps

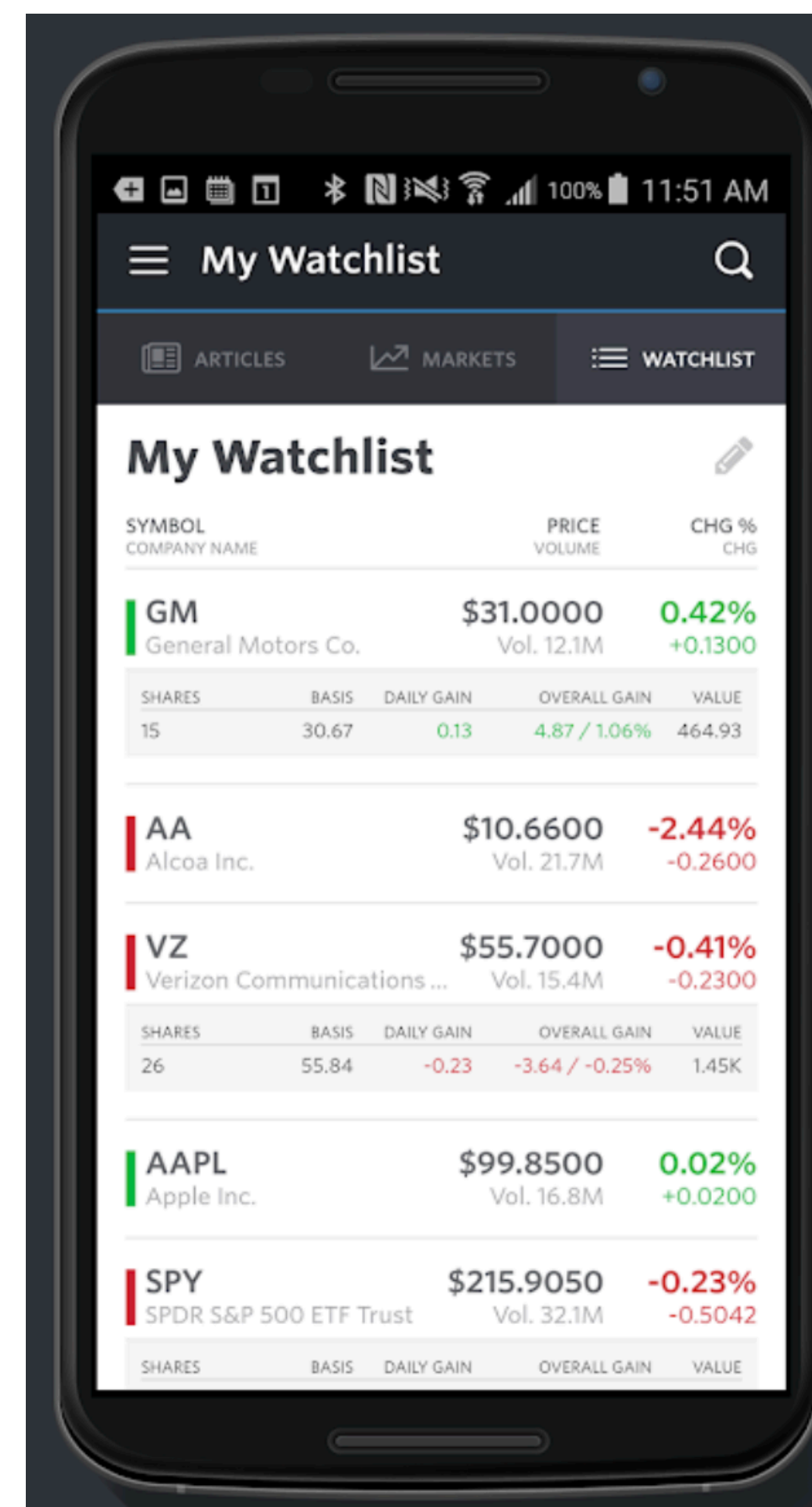
Hybrid-Web

Some examples of Ionic apps

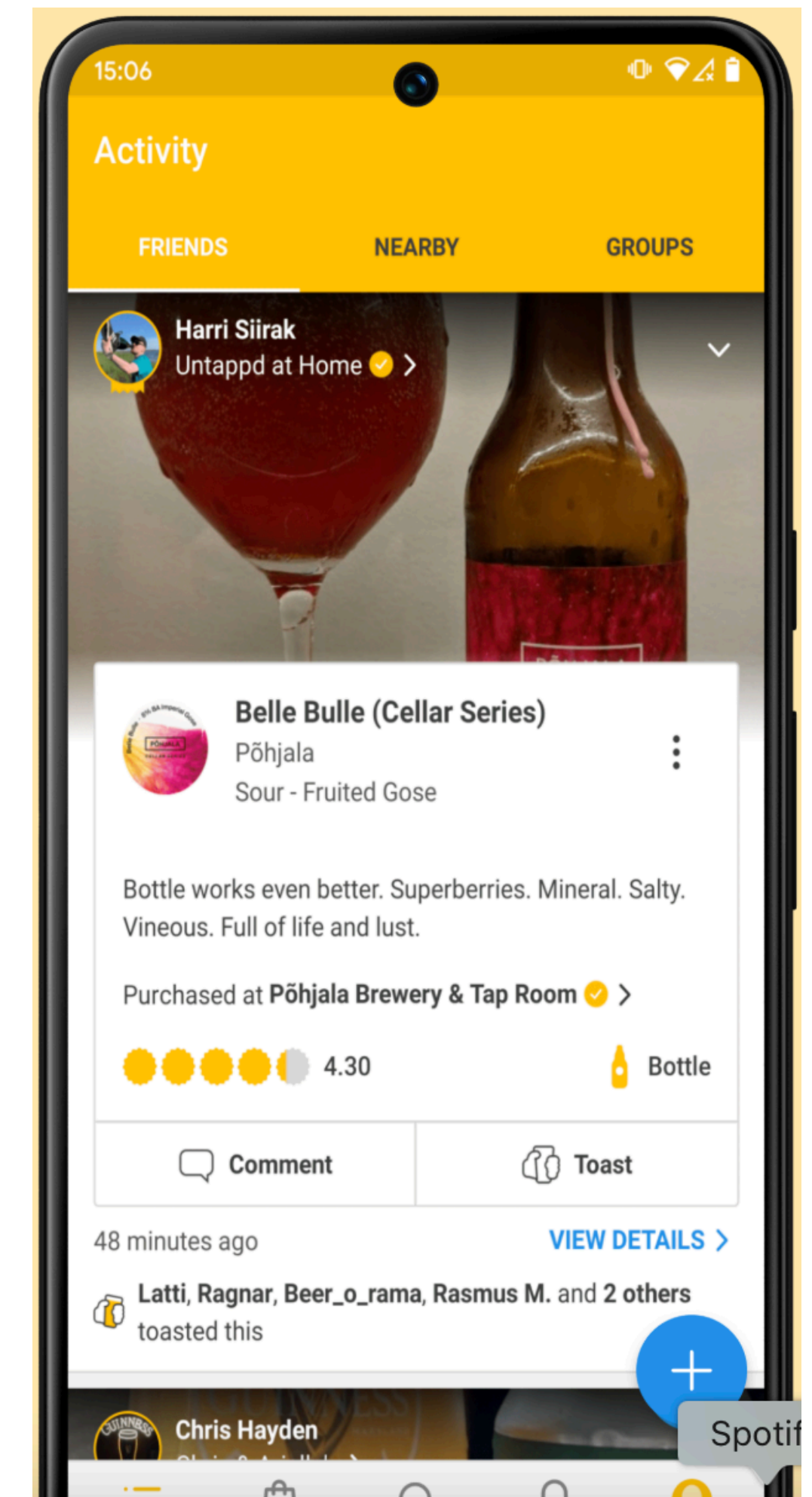
Burger King



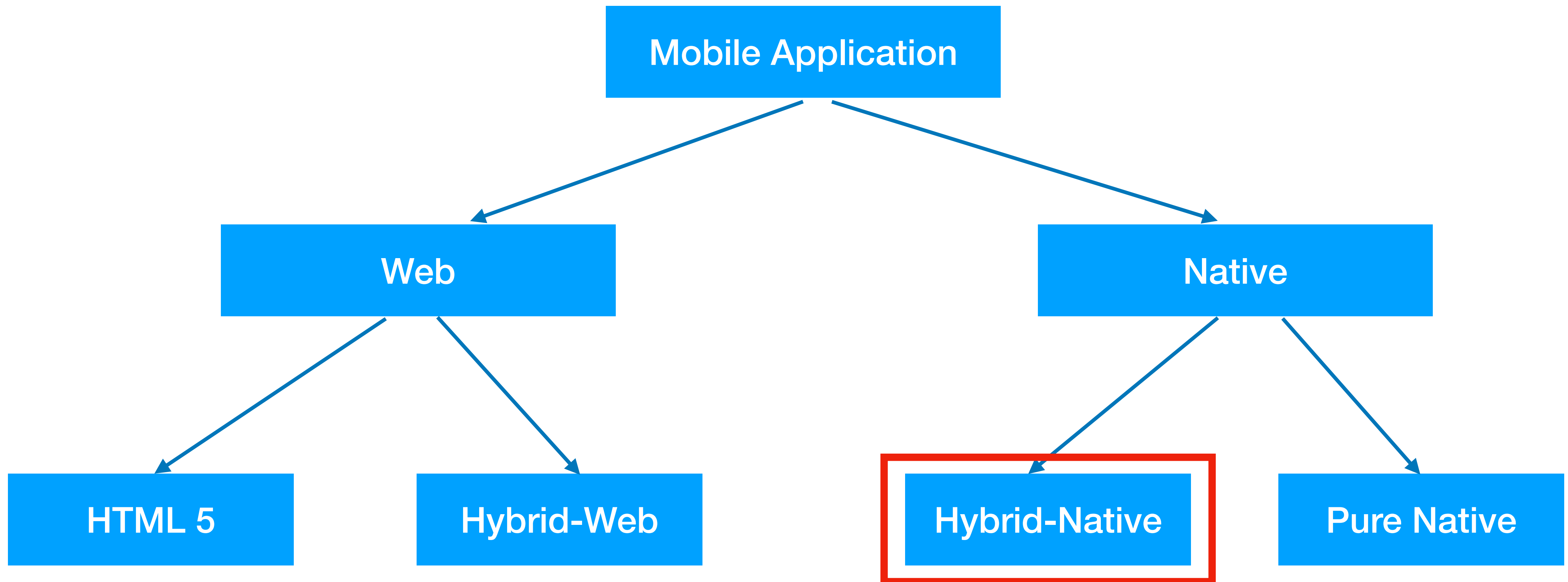
Market Watch



Untappd



Mobile apps



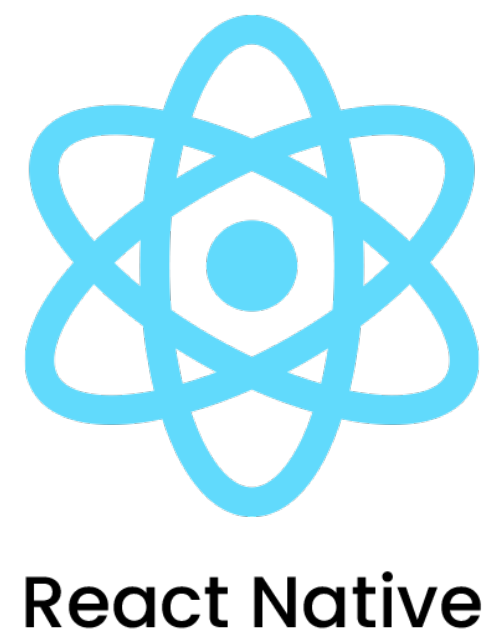
Mobile apps

Hybrid-Native

Hybrid-native apps:

- Are installable applications distributed through app stores
- Render the user interface using native UI components rather than a browser/WebView
- Enable cross-platform development with a shared codebase for iOS and Android (i.e., same programming language)

Examples: Appcelerator Titanium, React Native, Flutter, .NET MAUI (previously Xamarin), Unity (games), Kotlin Multiplatform



Mobile apps

Hybrid-Native

HTML5 / Hybrid-Web

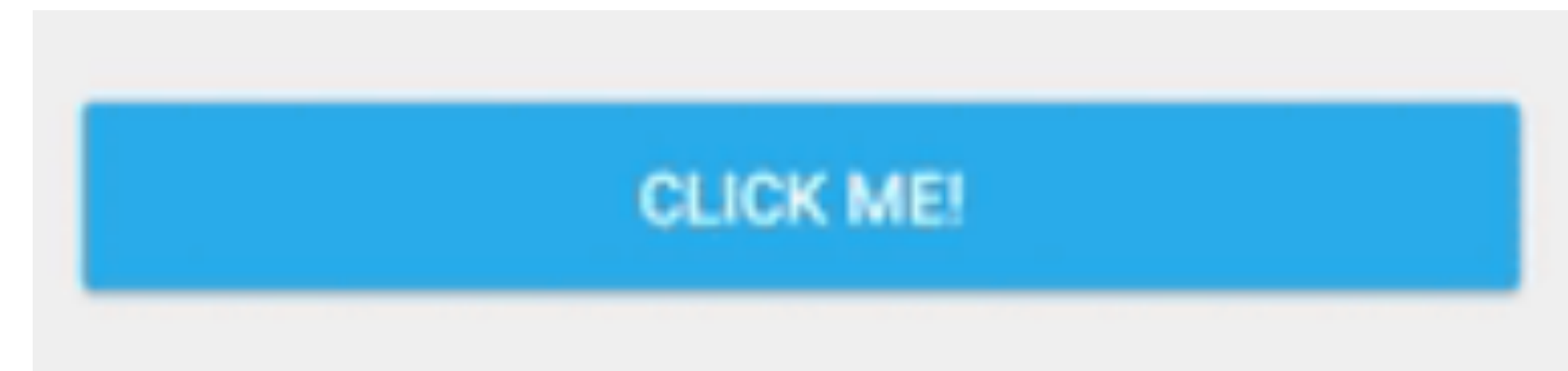


HTML

```
<button value="click me" />
```

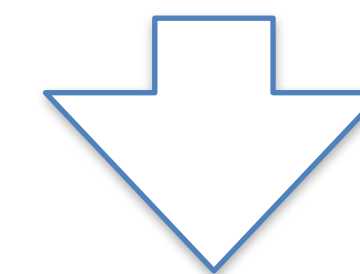
(interpreted by the browser to show a “virtual” button)

Hybrid-Native



React-Native

```
<ReactNative.Button title="click me" />
```



Android/Kotlin

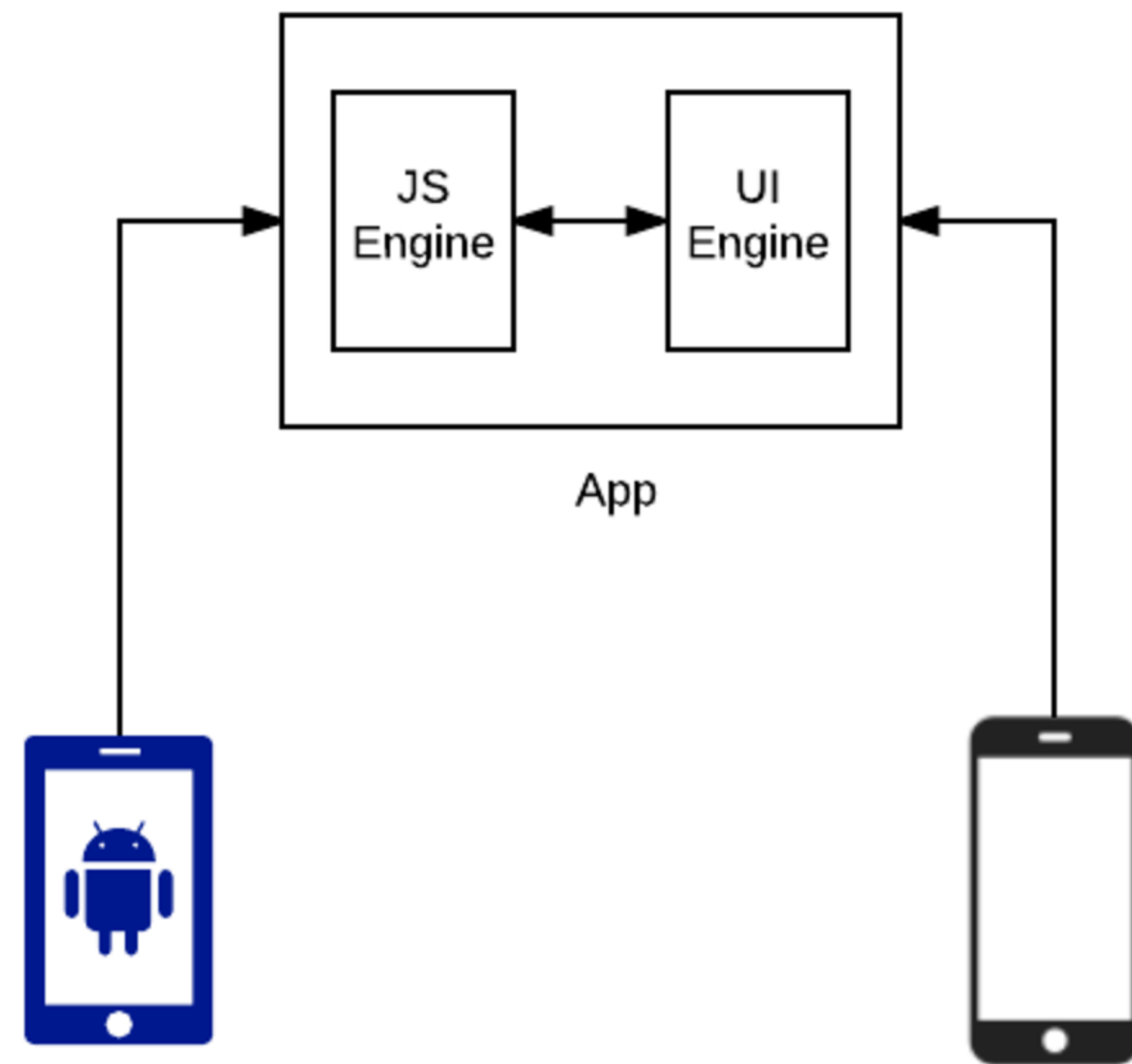
```
Button("click me")
```

(creates a real button through the Operating System API)

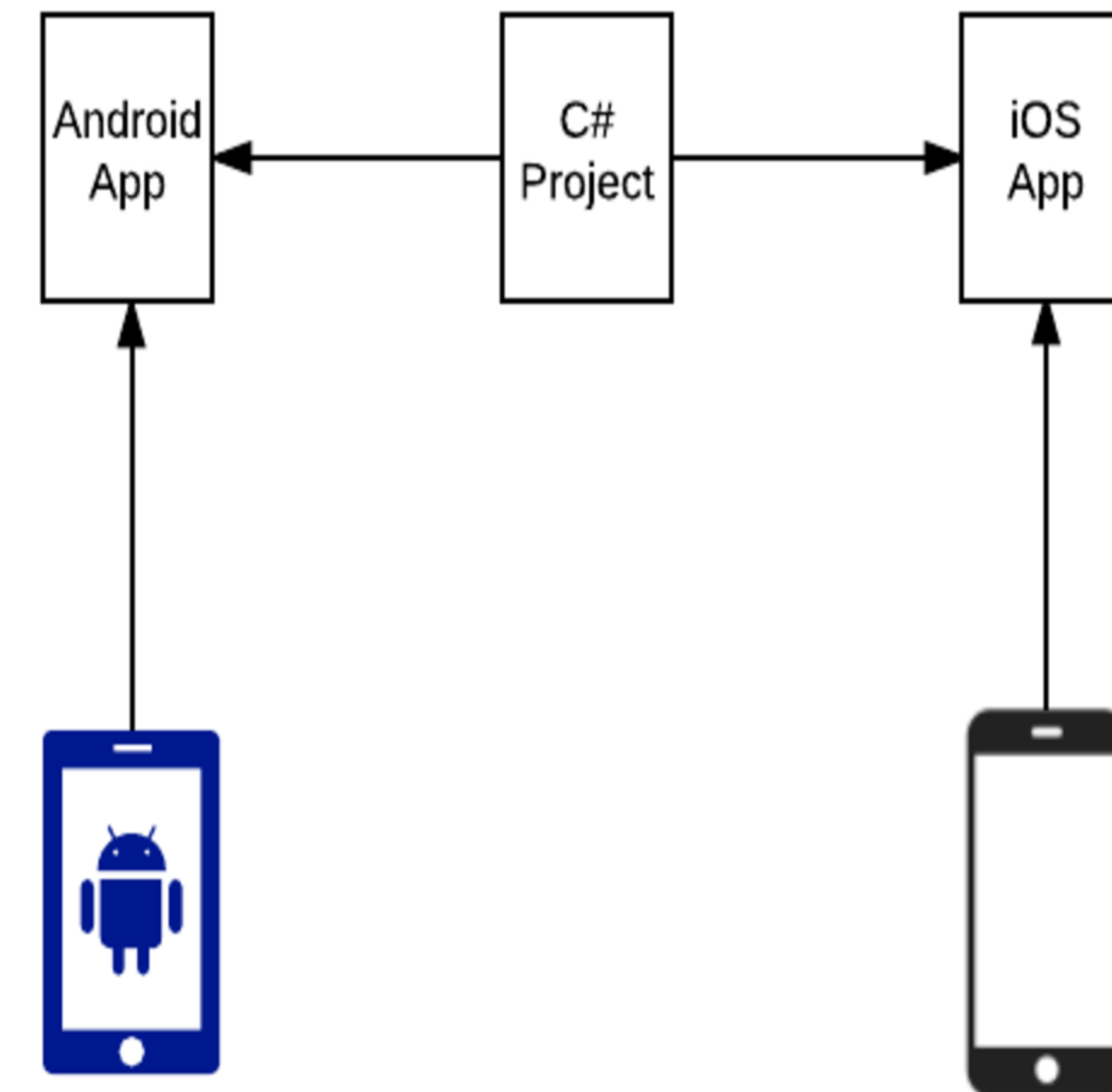
Mobile apps

Hybrid-Native

Hybrid-native apps translate cross-platform code to native-platform code using two techniques



Embedded runtime engine



Cross compiler

Mobile apps

Hybrid-Native

Embedded runtime engine

- Runs application logic inside an embedded runtime (e.g., JavaScript engine)
- UI components are transformed into native components during execution
- The runtime engine is bundled with the application

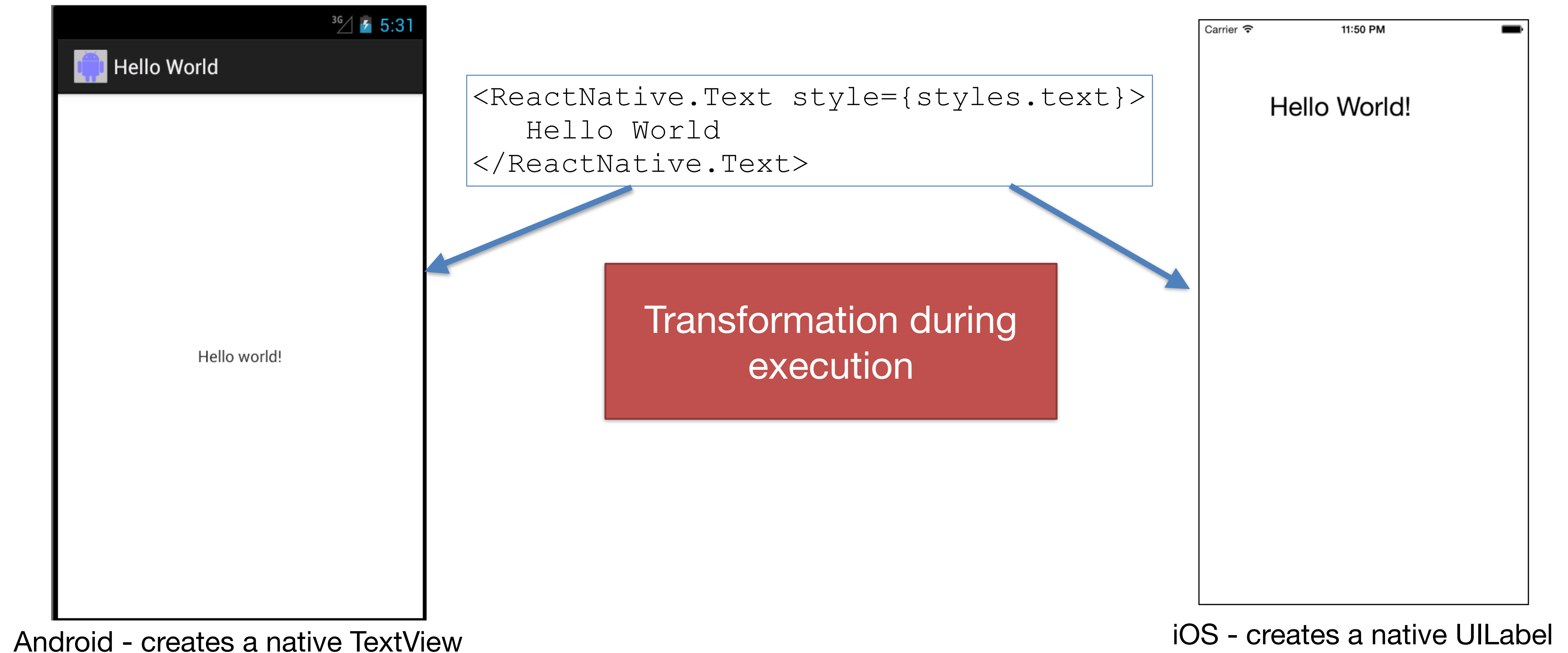
Cross compiler

- Compiles shared code into platform-native binaries
- UI components are transformed into native components during compilation

Mobile apps

Hybrid-Native

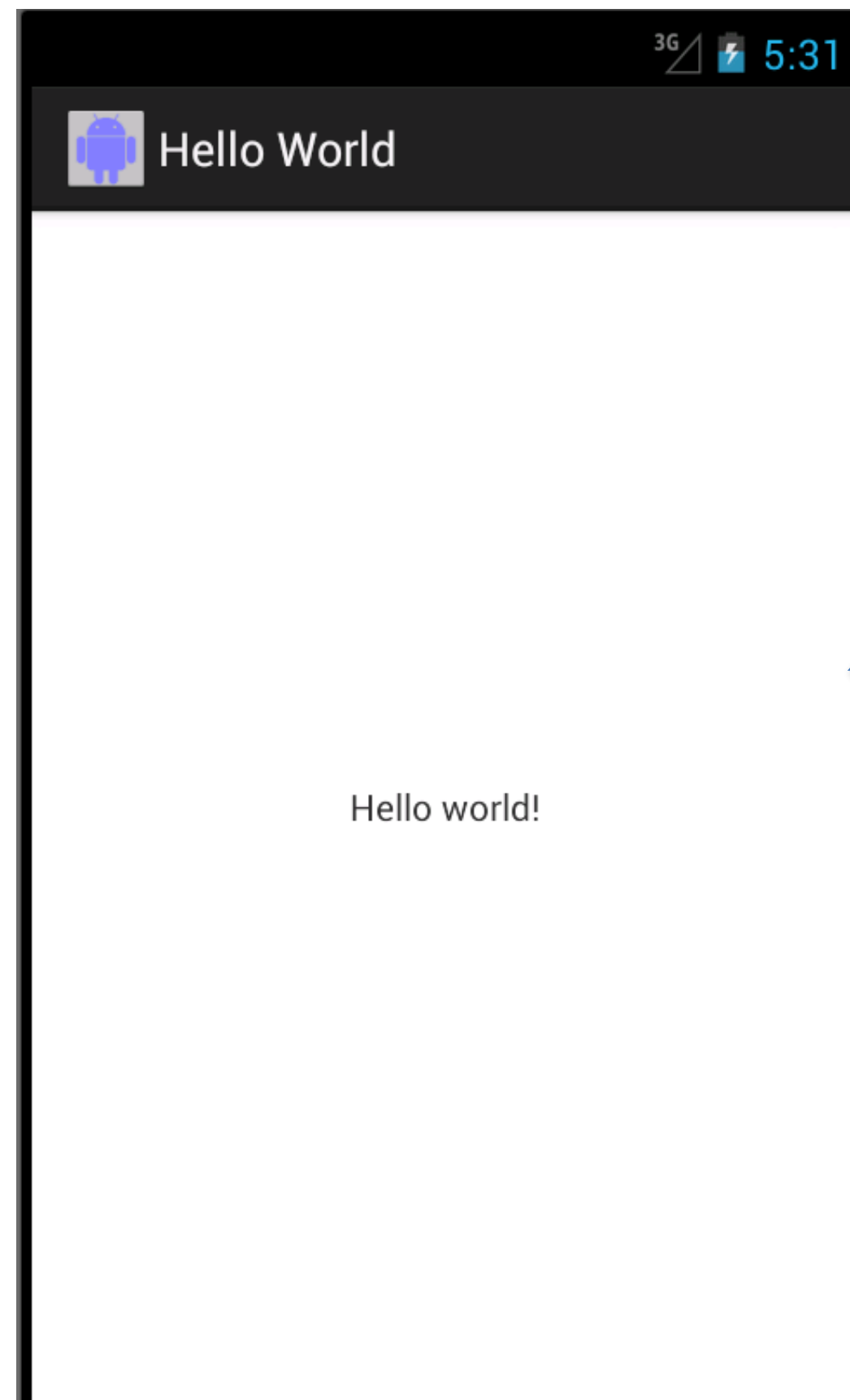
Embedded Runtime Engine (example in React Native)



Mobile apps

Hybrid-Native

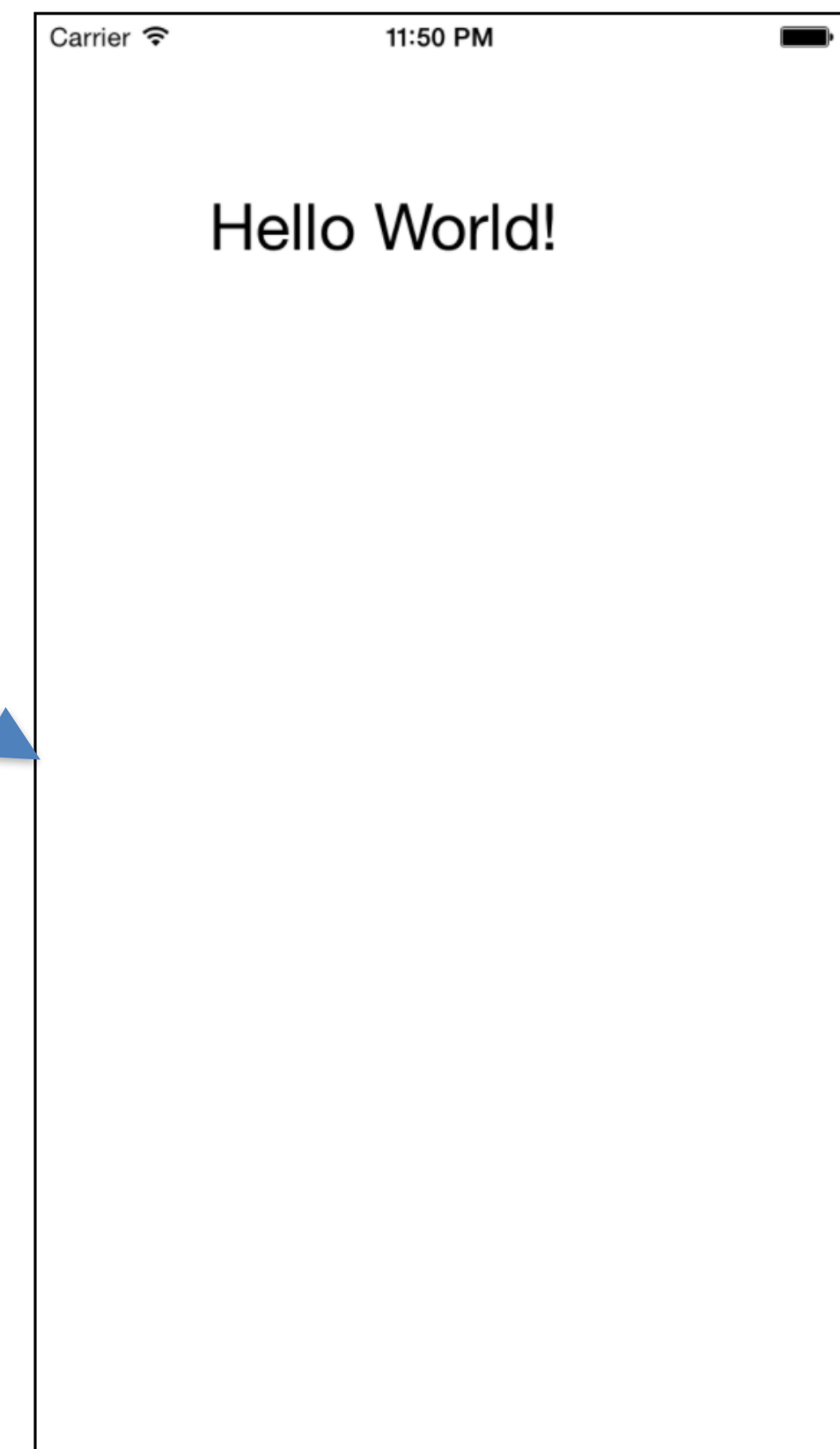
Cross compiler (example in .NET MAUI)



Android - creates a native TextView

```
<ContentPage>
  <Label Text="Hello, World!"
        FontSize="32"
        HorizontalOptions="Center"/>
</ContentPage>
```

Transformation during
compilation

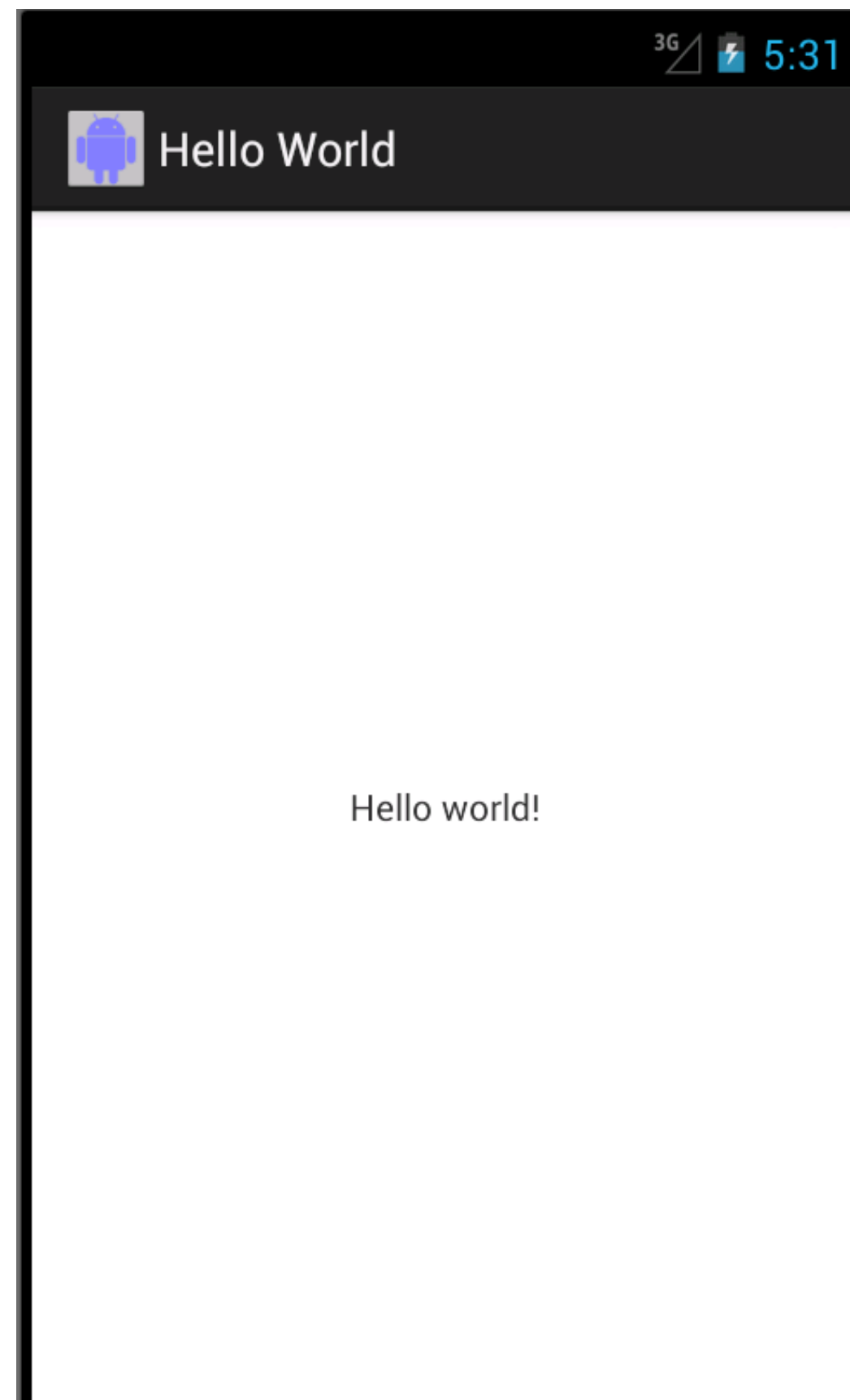


iOS - creates a native UILabel

Mobile apps

Hybrid-Native

Cross compiler (example in Flutter)



```
Widget build(BuildContext context) {  
  return MaterialApp(  
    title: 'Hello World',  
    home: Scaffold(  
      body: Center(  
        child: Text('Hello World'),  
      )),  
  );  
}
```

Transformation during
compilation

Android and iOS — creates a native
canvas where the label is rendered



Mobile apps

Hybrid-Native

Embedded runtime engine

- Since the UI is created at runtime, the first launch can be slower
- Allows (small) updates without going through App Store / Google Play
- Enables fast development (instant updates via Hot Reload)
- The binary is larger since it includes the runtime engine

Cross compiler

- Final compiled binary is indistinguishable from a real native app - performance is similar
- Updates must go through App Store / Google Play (the review process may take several days)

Common disadvantage: Dependency of non-official frameworks

Mobile apps

Hybrid-Native

Cross compiler

- Some Cross Compiler frameworks use a mixed approach:
 - **In development mode**, use an embedded runtime engine
 - **In production mode**, compile a native binary to avoid performance problems

Example: Flutter enabled Hot Reload in dev mode through an embedded Dart VM

Mobile apps

Hybrid-Native

Two approaches for rendering the UI components

- OEM - Use native components (e.g., Button, Label, ...)
- Custom - Use a native canvas, draw the component “by hand”

Mobile apps

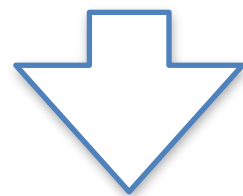
Hybrid-Native

Custom Widgets



Flutter

```
ElevatedButton(  
  child: Text("click me"),  
)
```



Android/Kotlin

```
Canvas(  
  drawBitmap(...)
```

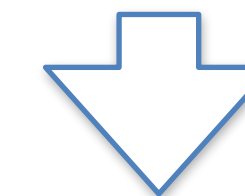
(creates a virtual button that is drawn on top of an empty canvas)

OEM Widgets



React-Native

```
<Button title="click me" />
```



Android/Kotlin

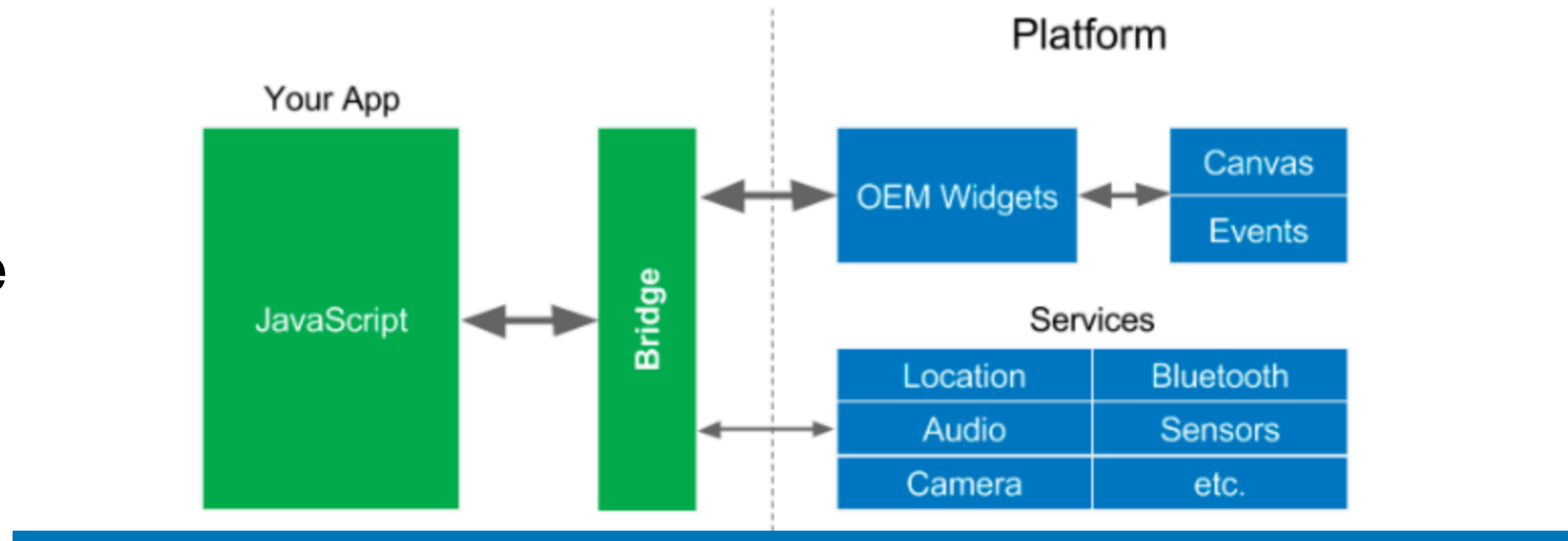
```
Button("click me")
```

(creates a real native button)

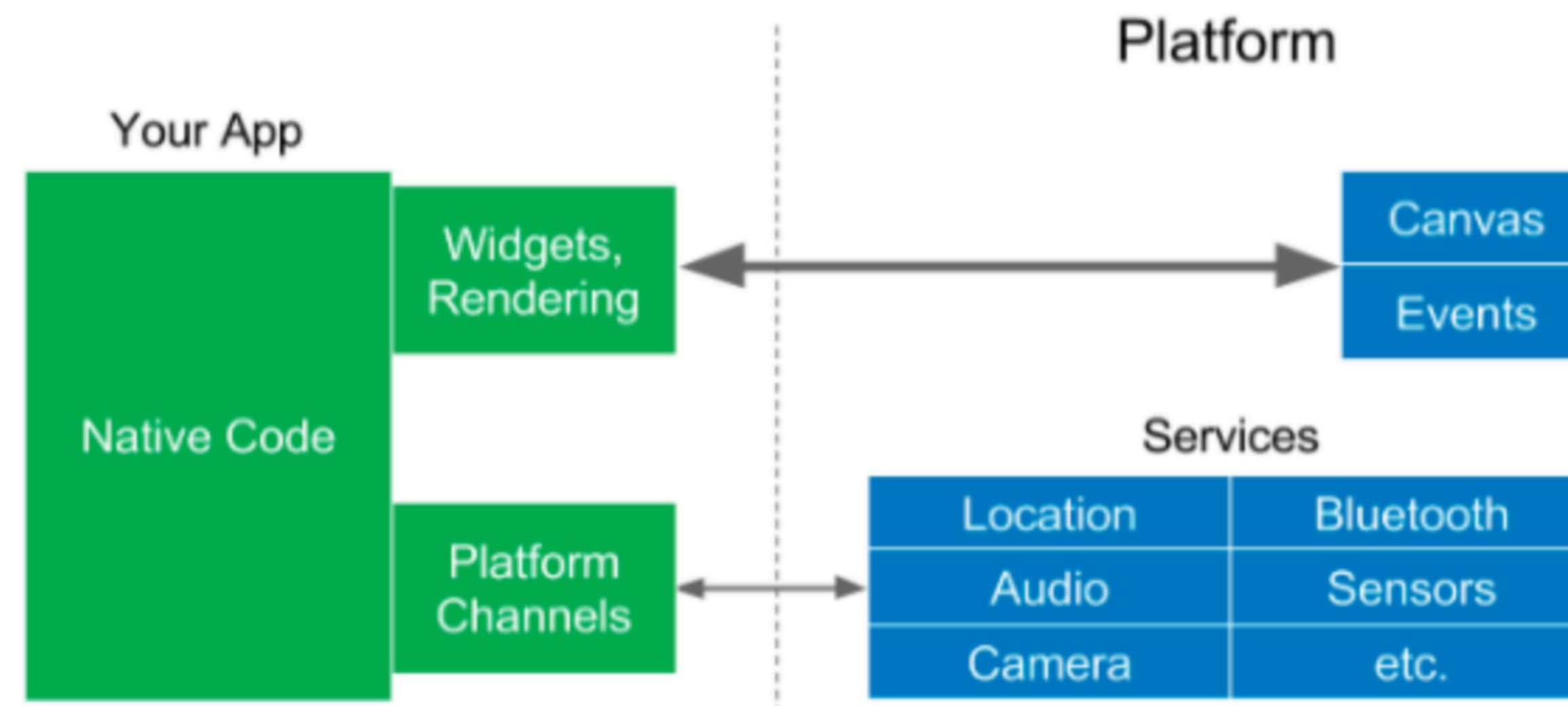
Mobile apps

Hybrid-Native

React Native



Flutter



Mobile apps

Hybrid-Native

Main frameworks

Framework	Start year	Company	Language	Technique	Widgets	Allows direct update?
Titanium	2008	Axway	Javascript	Runtime (JS) Engine	OEM	No
React Native	2015	Facebook	Javascript	Runtime (JS) Engine	OEM	Yes
Flutter	2015	Google	Dart	Runtime (Dart) Engine / Cross-compiler	Custom	Yes (**)
.NET MAUI	2022 (*)	Microsoft	C#	Cross-compiler	OEM	No
Kotlin Multiplatform (KMP)	2023	Jetbrains	Kotlin	Cross-compiler	OEM	No

(*) Before that, Microsoft offered Xamarin, discontinued in 2024

(**) Using shorebird.dev (paid)

Mobile apps

Hybrid-Native

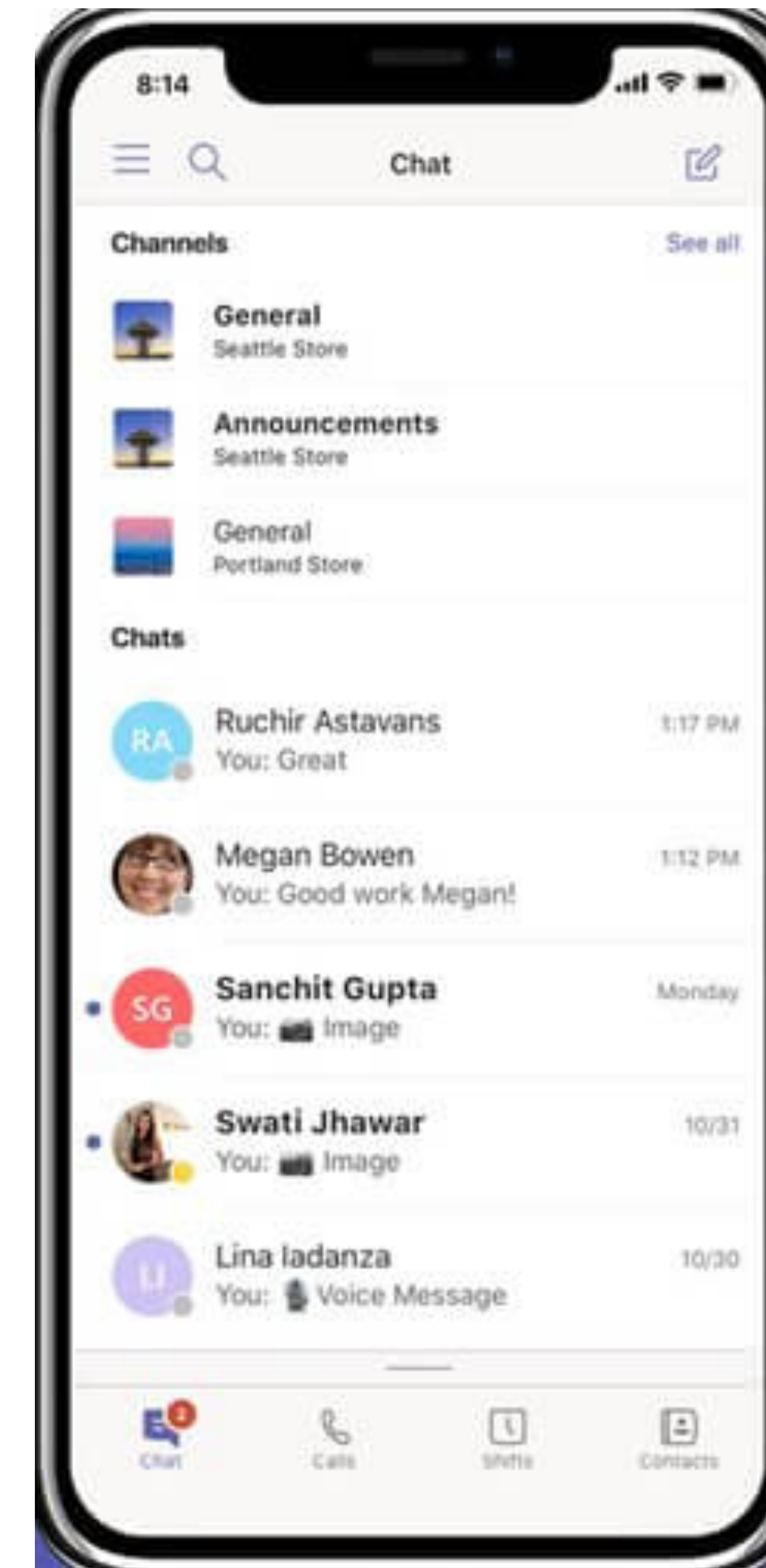
Some examples of React Native apps

more: <https://reactnative.dev/showcase>

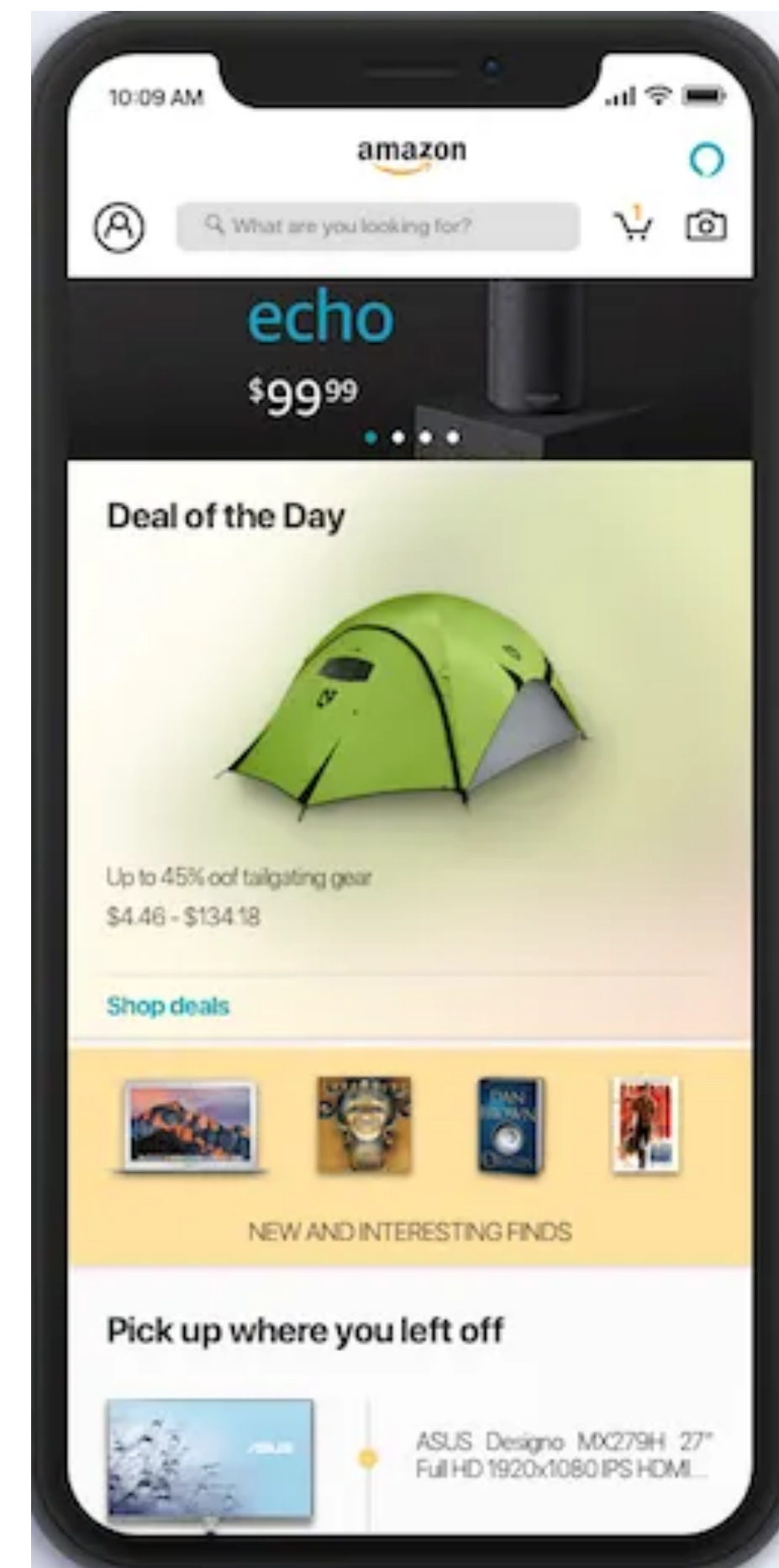
Facebook



Microsoft Teams



Amazon



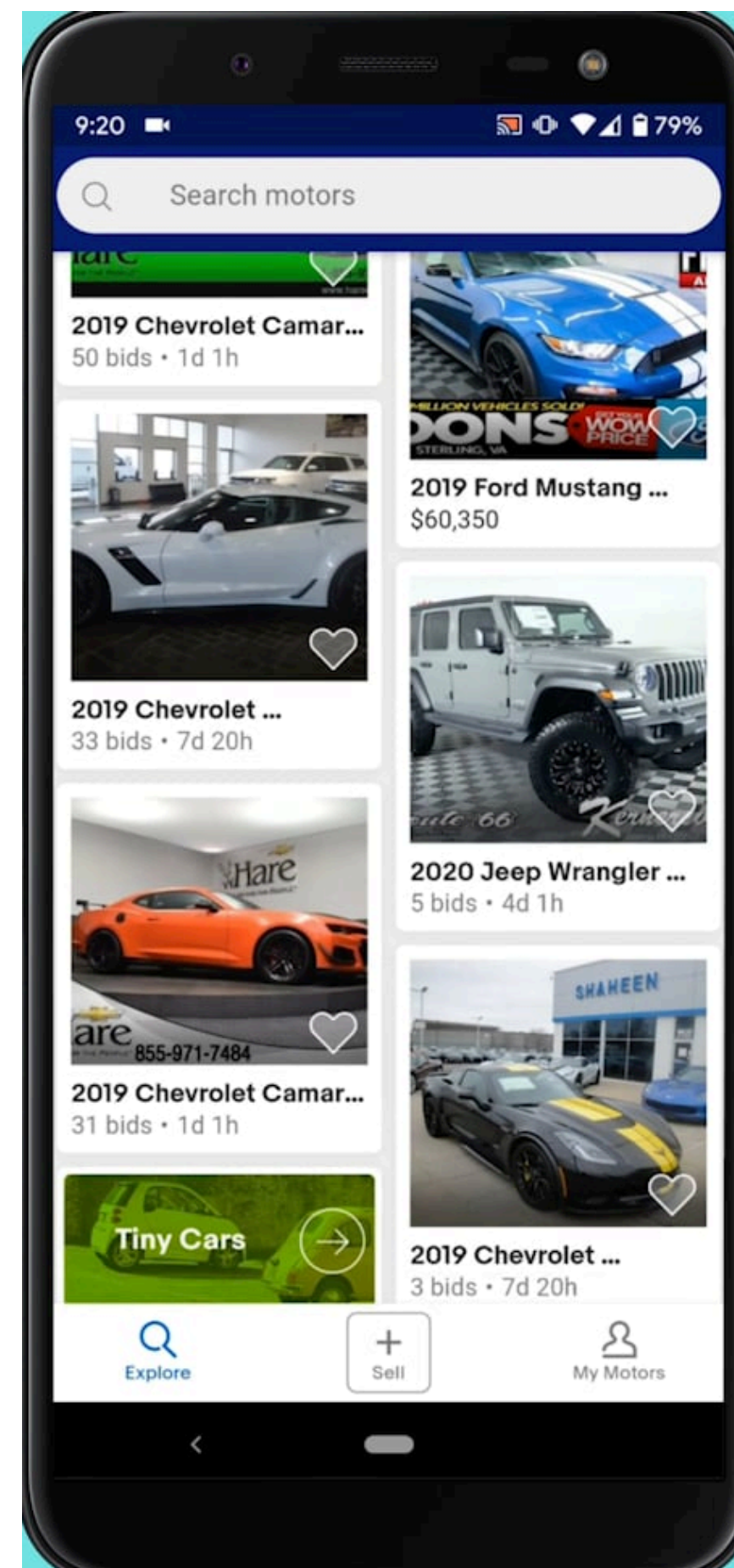
Mobile apps

Hybrid-Native

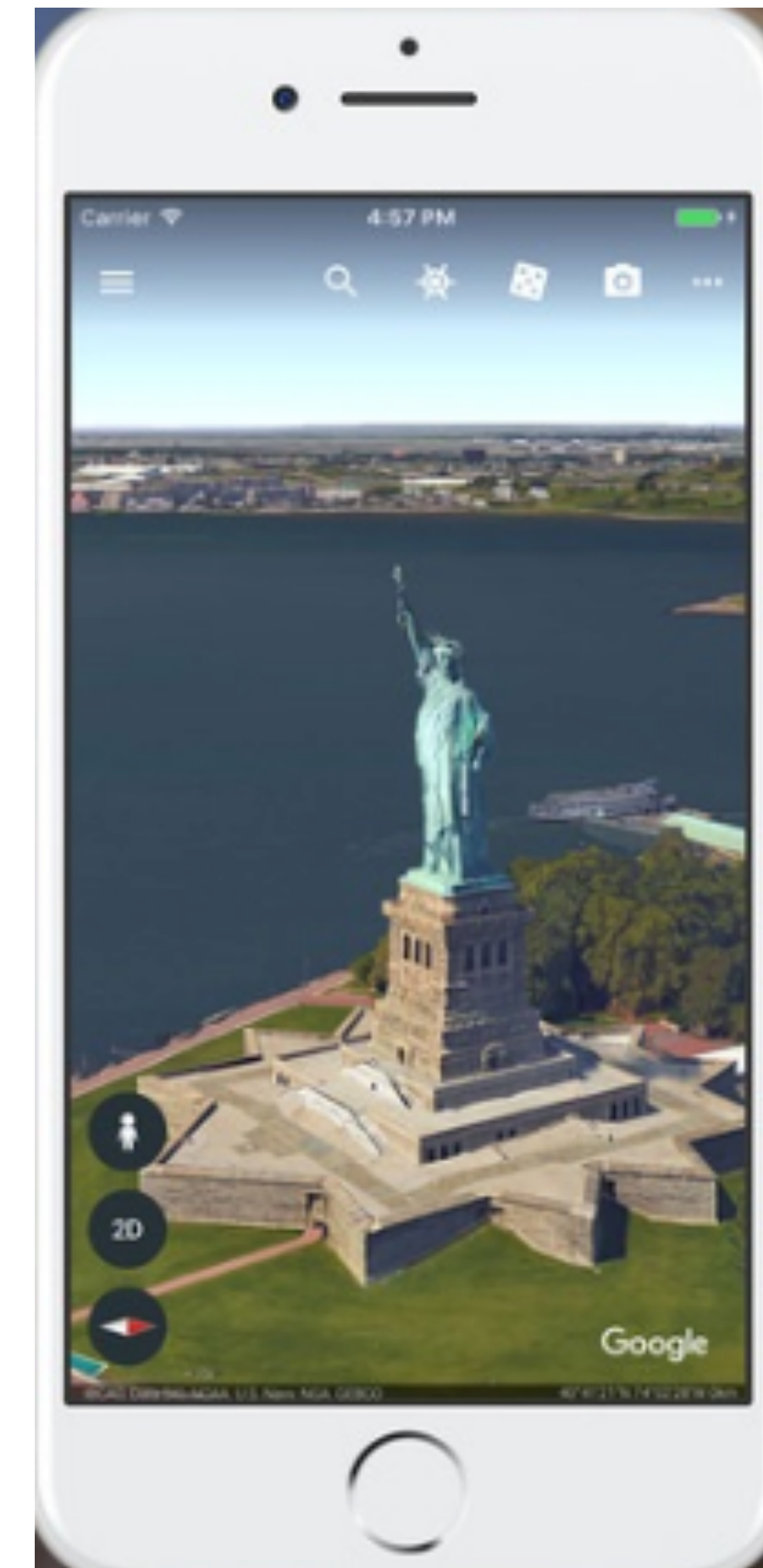
Some examples of Flutter apps

more: <https://flutter.dev/showcase>

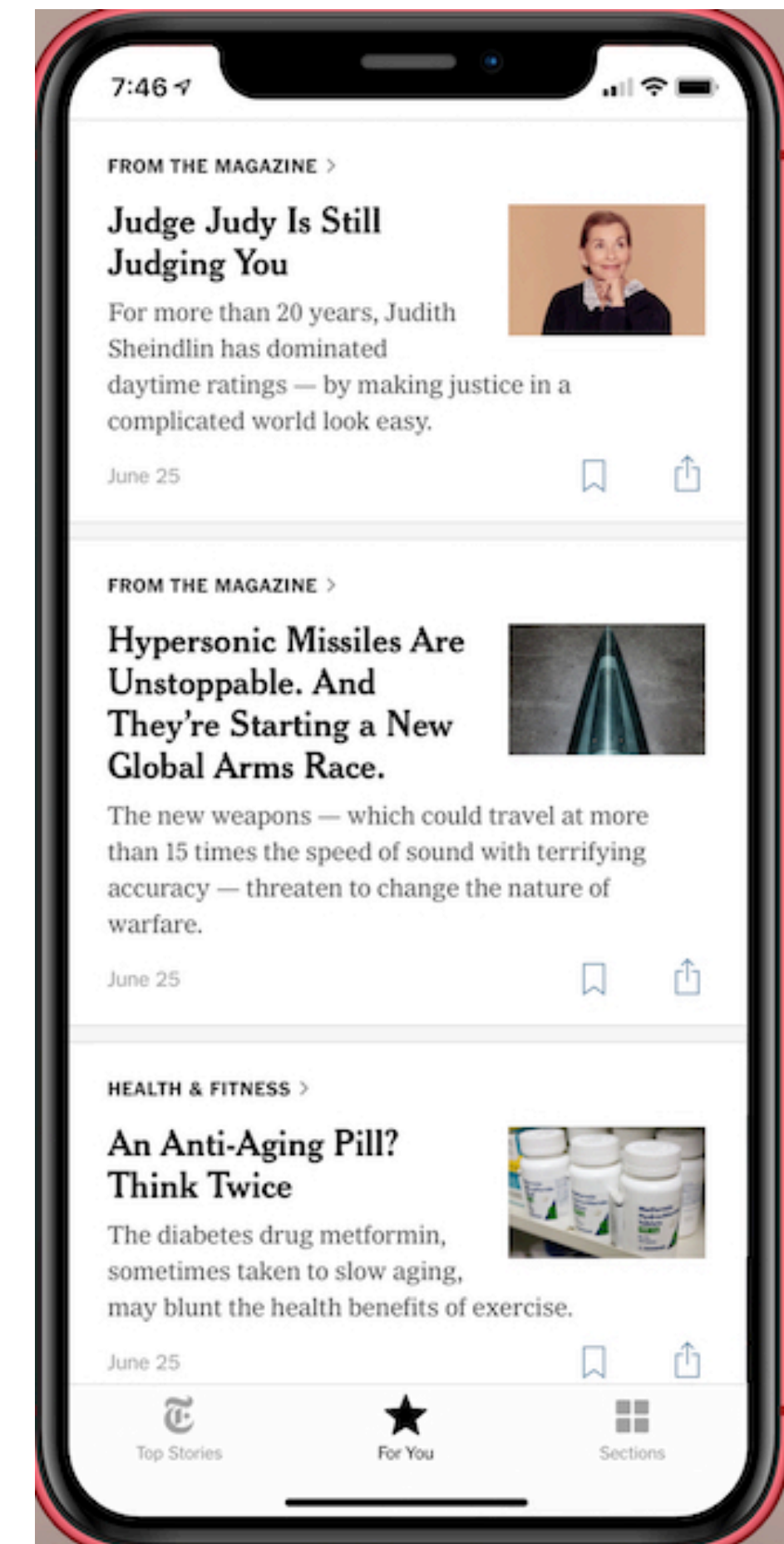
Ebay Motors



Google Earth



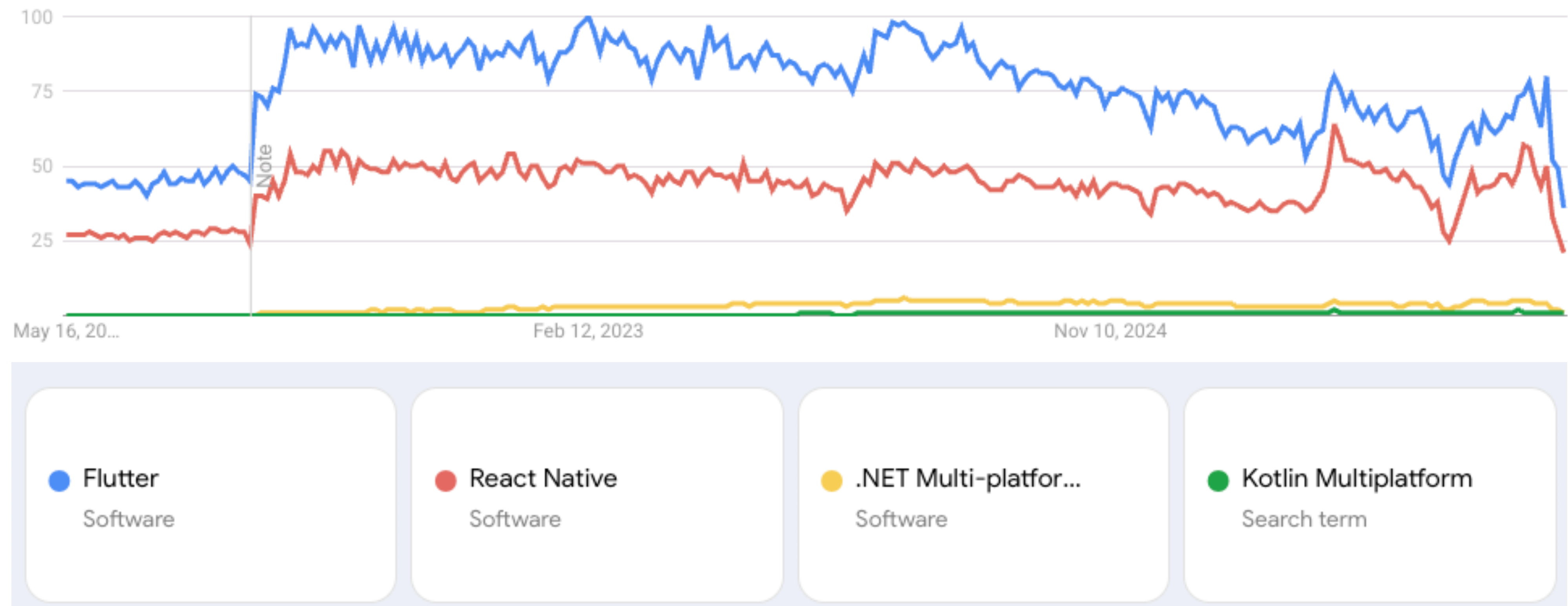
New York Times



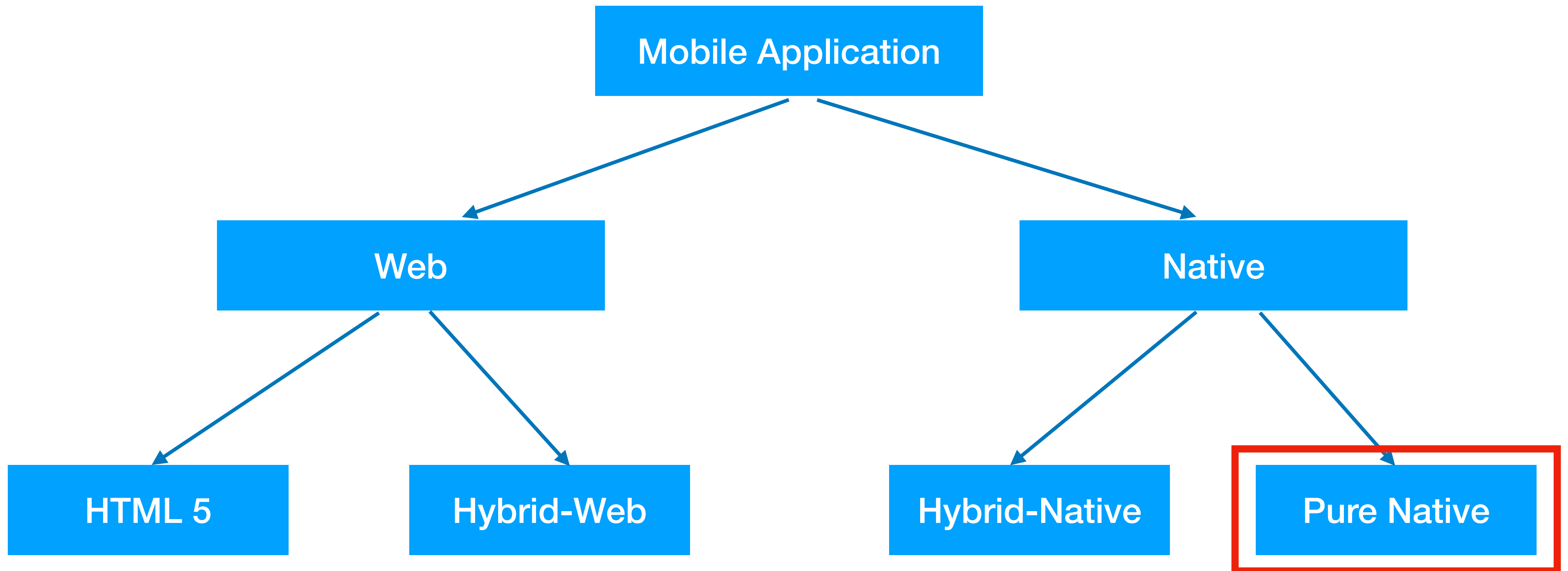
Mobile apps

Hybrid-Native

Google searches in the last 5 years



Mobile apps



Mobile apps

Native

We already know that development costs are duplicated

Advantages

- **Maximum flexibility/control:** If something cannot be developed in pure native, then it certainly cannot be done using any other approach
- **Best performance:** Other approaches (even hybrid-native) will always be at a disadvantage because they cannot achieve the same level of optimization (compiler, profiling)
- **Largest community:** Most mobile application developers work with this approach, so there is more information available online
- **Fewer bugs:** Bugs are less likely in the Android SDK and iOS SDK than in other frameworks

Exercise

	HTML5	Hybrid-Web	Hybrid-Native	Native
Development cost				
Access to device features				
User interface and navigation				
Smoothness / Performance				
Offline support				
Updates				

Resolution

	HTML5	Hybrid-Web	Hybrid-Native	Native
Development cost	Low	Low	Medium (1)	High
Access to device features	Limited	Medium (2)	Almost full (3)	Full
User interface and navigation	Web	Semi-native (4)	Same as the OS (5) / Custom (6)	Same as the OS
Smoothness / Performance	Low	Low	High / Medium-high (7)	High
Offline support	Does not work	Always works	Always works	Always works
Updates	Immediate	May take several days / Immediate (8)	May take several days / Immediate (9)	May take several days

Resolution

- (1) More sophisticated technology, requiring more expensive developers
- (2) Capacitor provide access to features that HTML5 alone does not support
- (3) Tool vendors usually take some time to integrate new features introduced in each operating system version
- (4) Ionic provides CSS that adapts the appearance to each OS
- (5) When using OEM widgets
- (6) When using custom widgets
- (7) When using an embedded runtime engine, transformations are performed during screen rendering (slower)
- (8) Using Ionic Appflow, updates can be immediate through Code Push
- (9) When using an embedded runtime engine, updates can be immediate through Code Push

Test 2

- It will have a duration of 1h10
- It will cover all material from lessons 9 through 11 (inclusive).
- You may not bring scratch paper or a cheat sheet (the study material is not extensive enough to justify it).
- The exam must be answered directly on the test paper, using pen or pencil.
- There will be a multiple-choice section (worth at most half of the total grade) covering CSR, React and Mobile. Incorrect answers will be penalized. Definitions may appear, but questions are more likely to focus on comparisons between approaches, best practices, etc. Small code snippets may also be included.
- There will be three open-ended questions about React, based on exercises such as the following:
 - Draw a react component tree for a given problem (see last slides of lecture 10)
 - Fill in the blanks in React code (components, react hooks, fetch api, react router)
 - Identify what is printed in the console by a given react component
 - Detect errors in React code excerpts (underline the line with error and justify)

Exercise

Spot the errors
For each one, justify

```
1. function getDefaultRating(): number {
2.     console.log("computing default rating")
3.     return 3
4. }
5.
6. export default function reviewsPage({ productId }: { productId: number }) {
7.     const [reviews, setReviews] = useState<Review[]>([])
8.     const [comment, setComment] = useState('')
9.     const [rating, setRating] = useState(getDefaultRating())
10.
11.     reviewService.getReviews(productId).then(setReviews)
12.
13.     function handleSubmit() {
14.         reviews.push({ id: Date.now(), comment, rating })
15.         setReviews(reviews)
16.         setComment('')
17.     }
18.
19.     return (
20.         <div>
21.             <textarea value={comment} />
22.             <select value={rating} onChange={e => setRating(e.target.valueAsNumber)}>
23.                 <option value={1}>1 star</option>
24.                 <option value={2}>2 stars</option>
25.                 <option value={3}>3 stars</option>
26.             </select>
27.             <button onClick={handleSubmit}>Submit</button>
28.             {reviews.map(r => <p>{r.comment} – {r.rating} stars</p>)}
29.         </div>
30.     )
31. }
```

Resolution

```
1. function getDefaultRating(): number {
2.     console.log("computing default rating")
3.     return 3
4. }
5.
6. export default function reviewsPage({ productId }: { productId: number }) { ← React components must start
7.     const [reviews, setReviews] = useState<Review[]>([])                                with uppercase
8.     const [comment, setComment] = useState('')
9.     const [rating, setRating] = useState(getDefaultRating()) ← should be useState(getDefaultRating)
10.
11.     reviewService.getReviews(productId).then(setReviews) ← should be inside a useEffect()
12.
13.     function handleSubmit() {
14.         reviews.push({ id: Date.now(), comment, rating }) ← mutates the existing state instead of creating a
15.         setReviews(reviews)                                new one
16.         setComment('')
17.     }
18.
19.     return (
20.         <div>
21.             <textarea value={comment} /> ← missing onChange that sets
22.             <select value={rating} onChange={e => setRating(e.target.valueAsNumber)}>    the comment
23.                 <option value={1}>1 star</option>
24.                 <option value={2}>2 stars</option>
25.                 <option value={3}>3 stars</option>
26.             </select>
27.             <button onClick={handleSubmit}>Submit</button>
28.             {reviews.map(r => <p>{r.comment} – {r.rating} stars</p>)}
29.         </div>
30.     )
31. }
```